



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Aplicación web para el cálculo
de trazas de herbivoría
mediante visión por
computador**



Presentado por Marcos Zamorano Lasso
en Universidad de Burgos — 8 de junio
de 2026

Tutor: Álgar Arnaiz González y David
Martínez Acha



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



El Dr. Álgvar Arnaiz González, junto a D. David Martínez Acha, profesor e investigador del departamento de Ingeniería Informática, área de Lenguajes y Sistemas informáticos.

Exponen:

Que el alumno D. Marcos Zamorano Lasso, con DNI 71482311J, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Aplicación web para el cálculo de trazas de herbivoría mediante visión por computador.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 8 de junio de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

Dr. Álgvar Arnaiz González

D. David Martínez Acha

Resumen

La monitorización de ecosistemas mediante imágenes aéreas permite analizar dinámicas espaciales asociadas a la presión ambiental, la degradación del terreno y la actividad de la fauna. En este contexto, las trazas generadas por el tránsito continuado de herbívoros constituyen un indicador visual relevante para estudiar patrones de uso del territorio. Sin embargo, su identificación manual en ortoimágenes resulta costosa, subjetiva y difícil de escalar, especialmente cuando se trabaja con zonas extensas o con estructuras lineales poco definidas.

Este proyecto aborda el desarrollo de una aplicación web para la detección y visualización automática de trazas de herbivoría mediante técnicas de visión por computador. Para ello, se parte de un modelo de segmentación semántica previamente entrenado, que permite procesar imágenes aéreas y obtener las trazas detectadas. La aplicación integra dicho modelo en un entorno accesible para el usuario final, permitiendo cargar imágenes, ejecutar el proceso de inferencia y superponer los resultados obtenidos sobre la imagen original.

Además del flujo básico de análisis sobre imágenes individuales, el sistema se ha diseñado con una arquitectura extensible orientada a su evolución hacia escenarios geoespaciales de mayor escala. En esta línea, se incorporan funcionalidades relacionadas con un visor cartográfico, gestión de colecciones de imágenes, administración de usuarios y control de modelos de inferencia. Como resultado, el proyecto proporciona un prototipo funcional que combina aprendizaje profundo, procesamiento de imágenes, persistencia de datos y visualización web para facilitar el análisis de trazas asociadas a la presión de herbivoría.

Descriptores

visión por computador, segmentación semántica, aprendizaje profundo, trazas de herbivoría, ortoimágenes, teledetección.

Abstract

Ecosystem monitoring through aerial imagery makes it possible to analyse spatial dynamics related to environmental pressure, land degradation and wildlife activity. In this context, trails generated by the continuous movement of herbivores constitute a relevant visual indicator for studying patterns of land use. However, their manual identification in orthoimages is costly, subjective and difficult to scale, especially when working with large areas or with subtle linear structures.

This project addresses the development of a web application for the automatic detection and visualisation of herbivory trails through computer vision techniques. To this end, the project builds upon a previously trained semantic segmentation model, capable of processing aerial images and obtaining a representation of the detected trails. The application integrates this model into an accessible environment for final users, allowing them to upload images, execute the inference process and overlay the obtained results on the original image.

In addition to the basic analysis workflow for individual images, the system has been designed with an extensible architecture aimed at future evolution towards larger-scale geospatial scenarios. In this regard, the application incorporates features related to a cartographic viewer, image collection management, user administration and inference model control. As a result, the project provides a functional prototype that combines deep learning, image processing, data persistence and web visualisation to support the analysis of trails associated with herbivory pressure.

Keywords

computer vision, semantic segmentation, deep learning, herbivory trails, orthoimages, remote sensing.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
2.1. Objetivos del software	3
2.2. Objetivos técnicos	4
3. Conceptos teóricos	7
3.1. Visión por computador	7
4. Técnicas y herramientas	23
4.1. Técnicas metodológicas	23
4.2. Herramientas y tecnologías empleadas	25
4.3. Comparativa de extensiones para VS Code y motor de la BBDD	31
5. Aspectos relevantes del desarrollo del proyecto	37
6. Trabajos relacionados	45
6.1. Artículos científicos relacionados	45
6.2. Herramientas Visuales para Imágenes Satelitales y Aéreas . .	50
7. Conclusiones y Líneas de trabajo futuras	55
7.1. Conclusiones	55

7.2. Líneas de trabajo futuras	56
Bibliografía	59

Índice de figuras

3.1. Ejemplo de segmentación semántica.	8
3.2. Interpretación geométrica de IoU.	21
6.1. Interfaz de la herramienta QGIS	51
6.2. Interfaz de USGS EarthExplorer	52
6.3. Interfaz de Copernicus Browser	54

Índice de tablas

4.1. Comparativa de ventajas e inconvenientes entre Bootstrap y Tailwind CSS.	34
4.2. Comparativa de motores y fuentes de datos geoespaciales.	35
4.3. Comparativa de extensiones para la gestión de bases de datos en Visual Studio Code	36
5.1. Comparativa por <i>fold</i> entre carga e inferencia usando modelos serializados con pickle y exportados a TorchScript.	39
6.1. Comparativa de trabajos relacionados según objetivo, datos, enfoque metodológico y relación con el presente TFG.	49

1. Introducción

La monitorización de ecosistemas a partir de imágenes aéreas se ha convertido en una herramienta clave para detectar dinámicas espaciales asociadas a la degradación ambiental y a la pérdida de biodiversidad [66, 49, 7]. En este contexto, las sendas de herbívoros, originadas por el pisoteo continuado y organizadas en redes de trazado complejo, constituyen un indicador relevante de actividad de herbivoría intensa y, por tanto, un potencial marcador de zonas de presión ecológica [15]. Sin embargo, su identificación manual en ortoimágenes resulta costosa, subjetiva y difícil de escalar [49, 15], mientras que aproximaciones automáticas clásicas tienden a depender de información topográfica adicional o a producir predicciones a nivel de parche, lo cual limita la delimitación precisa de la estructura y continuidad de las sendas [15]. Frente a ello, la segmentación semántica permite abordar el problema como una clasificación densa a nivel de píxel, incorporando contexto local y global para reconstruir de forma más fiel la geometría de estos patrones sobre imágenes RGB [39, 41].

Este Trabajo Fin de Grado parte del algoritmo descrito en el artículo base [15], que evalúa arquitecturas modernas de segmentación semántica para mapear sendas de herbívoros en imágenes aéreas y pone de manifiesto la viabilidad de detectar estas estructuras mediante modelos de aprendizaje profundo. A partir de dicha base, el objetivo principal del proyecto consiste en trasladar esta capacidad de segmentación a un entorno accesible para el usuario final. Esto será posible mediante el desarrollo de una aplicación web que permita cargar una imagen, ejecutar el proceso de inferencia y devolver una visualización interpretable, superponiendo y dibujando las sendas detectadas sobre la imagen original. La aplicación incorporará un flujo de uso orientado a la interacción iterativa, de modo que el usuario pueda eliminar la imagen procesada y repetir el procedimiento con nuevas

entradas, consolidando así un prototipo funcional que encapsule el *pipeline* completo, desde la recepción del dato hasta la presentación del resultado.

Adicionalmente, el trabajo se plantea con una proyección evolutiva hacia un escenario de mayor escalabilidad, donde la fuente de datos no sea únicamente una imagen aislada subida por el usuario, sino servicios cartográficos de alta resolución, como un entorno soportado por un visor cartográfico con recursos del Plan Nacional de Ortofotografía Aérea (PNOA) [32]. En esa extensión, el usuario seleccionará una zona de interés sobre el terreno y el sistema obtendrá la imagen correspondiente para segmentarla y representar automáticamente la red de sendas en el área escogida. Esto abre la puerta a funcionalidades complementarias como la descarga del resultado o una creación de colecciones de zonas donde cada una de ellas contará con varias imágenes para visualizar y segmentar por separado.

La memoria se organiza de la siguiente manera: en primer lugar, se presenta el marco teórico necesario para contextualizar la segmentación semántica y los elementos arquitectónicos que la sustentan; a continuación, se describen las arquitecturas consideradas y el enfoque metodológico adoptado para su utilización en el problema de detección de sendas; seguidamente, se detallan el diseño e implementación del sistema, incluyendo el *pipeline* de procesamiento, la construcción de la interfaz web y la estrategia de despliegue; posteriormente, se exponen las pruebas realizadas y los resultados obtenidos, junto con su análisis; finalmente, se recogen las conclusiones y se definen líneas de trabajo futuro orientadas a la explotación de fuentes geoespaciales y a la ampliación de funcionalidades. Como materiales entregables complementarios a la memoria, se incluyen el código fuente del prototipo junto con su correspondiente *mockup*, los recursos necesarios para su ejecución y una guía de uso y reproducción del entorno, con el fin de facilitar la evaluación, la mantenibilidad y la continuidad del proyecto.

2. Objetivos del proyecto

Este apartado concreta los objetivos que se persiguen con la realización del proyecto, distinguiendo entre aquellos derivados de los requisitos del software a construir y los objetivos de carácter técnico necesarios para materializar la propuesta. En conjunto, el trabajo pretende trasladar a un contexto de uso real la capacidad de detectar sendas de herbívoros en ortoimágenes mediante segmentación semántica, apoyándose en el enfoque descrito en el artículo base [15], y encapsulando el proceso completo en una aplicación web orientada al usuario final.

De este modo, se busca no solo demostrar la viabilidad del *pipeline* de inferencia y visualización sobre imágenes aportadas por el usuario, sino también lograr una evolución posterior hacia fuentes geoespaciales de mayor escala, como Google Earth Engine [23].

2.1. Objetivos del software

Los objetivos del software se formulan como metas funcionales y de experiencia de uso. Se centran en definir qué capacidades debe ofrecer la aplicación y qué flujo de interacción debe soportar para que la detección automática de sendas sea accesible, interpretable y reutilizable en múltiples ejecuciones.

- Carga de imagen para predicción: permitir al usuario cargar una imagen aérea, preferentemente en formato estándar y ampliamente soportado, y validar el fichero antes de su procesamiento.
- Ejecución del proceso de inferencia: incorporar el *pipeline* proporcionado por el equipo docente [15] como motor de inferencia de la

aplicación, habilitando la carga de la imagen aportada por el usuario, el preprocesado compatible con el modelo y la obtención de la máscara de salida, incluyendo la gestión de predicción con varios modelos por *fold* cuando proceda.

- Postprocesado orientado a la interpretabilidad: adaptar y extender las rutinas de visualización ya incluidas en el *notebook* [15], integrándolas en la interfaz web, de modo que la superposición de la máscara y el resaltado de sendas sean claramente visibles para el usuario.
- Modo de inferencia y control de salida: integrar opciones de ejecución que permitan elegir un modo de inferencia y parametrizar la salida visual.
- Visualización interactiva del resultado: proporcionar una interfaz web que muestre la imagen de entrada y el resultado segmentado de manera sincronizada, habilitando operaciones de inspección como zoom, desplazamiento y ajuste de opacidad de la superposición.
- Flujo de trabajo iterativo: permitir que el usuario elimine la imagen procesada y repita el procedimiento con nuevas entradas.
- Gestión básica del resultado: permitir la descarga local de la imagen resultante con la superposición generada.
- Proyección funcional hacia escenarios geoespaciales: lograr una evolución funcional en la que la entrada sea un área seleccionada en un visor cartográfico, permitiendo la obtención automática de la ortoimagen, su segmentación y la visualización de la red de sendas resultante [32].

2.2. Objetivos técnicos

Los objetivos técnicos describen las decisiones de ingeniería necesarias para garantizar que el sistema sea mantenible, reproducible y eficiente, además de suficientemente flexible como para escalar hacia fuentes geoespaciales y flujos de trabajo más exigentes.

- Integración del *pipeline* de inferencia: adaptar el *pipeline* proporcionado [15], inicialmente desarrollado como *notebook*, a un componente software, desacoplado de la interfaz y con una ejecución replicable.

- Modularización de la lógica de inferencia: extraer la lógica de inferencia y visualización a un módulo Python con interfaces claras, apto para su invocación desde el *backend*.
- Diseño de una arquitectura web modular: definir una arquitectura clara con separación entre interfaz de usuario, *backend* de servicios y módulo de inferencia.
- Definición de una API de procesamiento: diseñar una API que permita la carga de imágenes, la ejecución de la inferencia, la consulta del estado del procesamiento y la recuperación de resultados.
- Gestión del cómputo y recursos: considerar el impacto computacional de la ejecución de múltiples modelos y definir mecanismos básicos de control del uso de recursos.
- Persistencia temporal de datos: definir una estrategia de almacenamiento temporal para imágenes y resultados, considerando una eliminación tras su uso.
- Trazabilidad del *pipeline*: registrar información básica de cada ejecución, como parámetros utilizados, versión del modelo y tiempos de cómputo.
- Integración con fuentes geoespaciales: diseñar el sistema de forma que permita sustituir la carga manual de imágenes por la obtención automática desde fuentes geoespaciales [47, 1].
- Despliegue y reproducibilidad: empaquetar la aplicación con una configuración de entorno reproducible y documentación básica de instalación y ejecución.

3. Conceptos teóricos

Este capítulo presenta el marco teórico que sustenta el desarrollo del proyecto y sirve como base para interpretar, con rigor, las decisiones metodológicas y los resultados obtenidos. Para ello, se organiza en tres bloques principales: en primer lugar, se introduce el contexto general en el que se encuadra el problema, conectando la representación visual con las tareas de análisis automático; a continuación, se desarrollan los fundamentos específicos de la segmentación semántica, incluyendo los elementos arquitectónicos y las estrategias de diseño más relevantes en modelos de predicción densa; finalmente, se recopilan los criterios y métricas empleados para evaluar el rendimiento, de modo que el capítulo deje establecida una terminología homogénea y un marco comparativo consistente para los capítulos posteriores.

3.1. Visión por computador

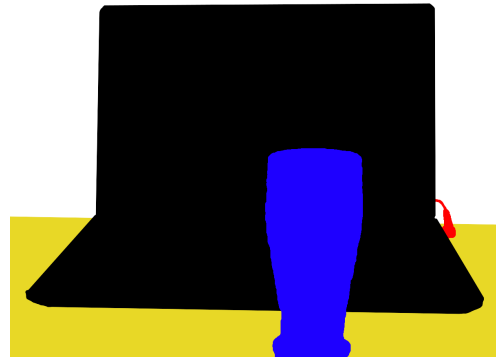
La Visión por Computador es la rama de la Inteligencia Artificial que estudia cómo dotar a un sistema de la capacidad de interpretar imágenes y vídeo, de modo que una matriz de píxeles se transforme en información estructurada y accionable sobre el entorno, como categorías, localizaciones, contornos u otras magnitudes geométricas relevantes.

Esta disciplina se apoya, por un lado, en la modelización del proceso de formación de imagen y de la geometría de proyección, lo cual permite relacionar el mundo tridimensional con el plano imagen. Por otro lado, operacionaliza ese conocimiento mediante un *pipeline* de procesamiento que convierte progresivamente la señal visual en representaciones más informativas, encadenando transformaciones que filtran, realzan y reorganizan la información hasta hacerla útil para una tarea concreta.

Sobre estas representaciones, el sistema ejecuta tareas como clasificación, detección o segmentación, y en particular la segmentación semántica permite asignar una etiqueta a cada región de la imagen preservando estructura espacial, tal y como ejemplifica el proceso de la Figura 3.1, conectando de forma natural con los fundamentos que se desarrollan en las secciones posteriores.



(a) Ejemplo para segmentación.



(b) Ejemplo de segmentación con regiones diferenciadas.

Figura 3.1: Ejemplo de segmentación semántica: (a) imagen original y (b) máscara segmentada. Fuente: MartinThoma. Licencia: CC0 1.0.

Conceptos teóricos de la Segmentación Semántica

En esta sección se introducen los fundamentos teóricos que sustentan la segmentación semántica, abordándola desde la perspectiva del aprendizaje profundo y de las arquitecturas que permiten realizar predicción densa a nivel de píxel. Se describen los principios que rigen el uso de redes neuronales profundas, haciendo especial énfasis en el papel de las estructuras ultra-profundas y de los mecanismos que facilitan su entrenamiento, como las conexiones residuales y las *skip connections*. Asimismo, se analizan los componentes operativos esenciales de los modelos de codificador-decodificador, incluyendo conceptos clave como *kernels*, *stride*, *downsampling* y *upsampling*, los cuales determinan el compromiso entre resolución espacial y riqueza semántica. Finalmente, se presentan las dos familias arquitectónicas predominantes en este ámbito, las redes neuronales convolucionales y los *Vision Transformers*, destacando sus principios de funcionamiento y su relevancia en tareas de segmentación semántica de alta precisión.

Aprendizaje Profundo

El Aprendizaje Profundo es un paradigma de *machine learning* basado en el uso de redes neuronales convolucionales (CNN) o arquitecturas alternativas como los *Transformers*, que han surgido como el enfoque dominante para el reconocimiento visual [28, 30]. Funciona de la siguiente manera: un modelo muy flexible, formado por varias capas de operaciones con parámetros y funciones no lineales, aprende a extraer representaciones jerárquicas de los datos ajustando sus parámetros para minimizar el error en los ejemplos de entrenamiento.

El éxito del Aprendizaje Profundo se atribuye a su capacidad para trabajar con arquitecturas muy grandes y profundas, con millones de parámetros, logrando avances significativos en el reconocimiento de imágenes [53, 15, 45, 28]. Este resuelve la necesidad de los sistemas de visión de extraer la representación de características más efectiva a partir de datos complejos [45], automatizando lo que antes requería el diseño manual de características.

Estructuras ultra-profundas

En el marco del Aprendizaje Profundo, la profundidad de las representaciones y de la red es de importancia crucial para muchas tareas de reconocimiento visual [28, 30]. Las arquitecturas modernas han superado significativamente las redes iniciales, con modelos que han rebasado la barrera de las cien capas [30]. Estas estructuras ultra-profundas buscan enriquecer los niveles de características mediante el aumento del número de capas apiladas [28, 30].

Sin embargo, el aumento de la profundidad introduce un desafío conocido como el problema de la degradación. Esta ocurre cuando, al aumentar las capas apiladas, la precisión del entrenamiento se deteriora. Teóricamente, un modelo más profundo no debería tener un error de entrenamiento mayor que su contraparte menos profunda. El hecho de que esto suceda indica que los algoritmos de optimización actuales tienen dificultades para encontrar soluciones que optimicen estas estructuras ultra-profundas [28].

Esto se debe a que, a medida que la información o el gradiente pasan por muchas capas, pueden desvanecerse antes de alcanzar el inicio o el final de la red [30].

Predicción Densa (*Dense prediction*)

La predicción densa es una formulación de inferencia estructurada en la que el sistema estima, para cada ubicación del dominio espacial de la entrada, una variable de salida perteneciente a un espacio de etiquetas o a un espacio continuo [74]. En visión por computador abarca tanto clasificación por elemento, como regresión por elemento, dando lugar a campos de salida con fuerte dependencia local y coherencia geométrica [76, 37].

Desde el punto de vista inductivo, estas tareas se benefician de la *equivarianza traslacional* propia de arquitecturas totalmente convolucionales, del acoplamiento multi-escala para capturar contexto y del control explícito del *output stride* para balancear resolución y campo receptivo [38, 35]. La naturaleza densa impone consideraciones de regularidad de frontera, conservación de detalles de alta frecuencia, manejo del desbalanceo de clases y robustez frente a los problemas introducidos por *downsampling* [6, 71, 34].

Operativamente, la predicción densa se implementa con *encoders-decoders* convolucionales que combinan *downsampling* para agregar contexto y *upsampling* para reconstruir resolución, asistidos por *skip connections* por concatenación para preservar información de detalle [53, 4, 3].

Segmentación Semántica

La segmentación semántica es una tarea de predicción densa en Visión por Computador cuyo objetivo es la clasificación a nivel de píxel [4, 34], mediante la asignación de una etiqueta de clase semántica a cada píxel de una imagen de entrada. A diferencia de tareas como la clasificación, la segmentación semántica proporciona una comprensión completa del escenario, prediciendo así la categoría, ubicación y forma de cada elemento.

Este proceso típicamente se implementa a través de una arquitectura de red neuronal profunda de codificador-decodificador, llamada *encoder-decoder* de manera tradicional [34, 45, 4].

1. Codificación mediante encoder: El codificador, a menudo basado en una Red Neuronal Convolucional (CNN) [17] o un *Vision Transformer (ViT)* [4], toma la imagen de entrada y transforma los datos de esta en una representación de características de alto nivel mediante una reducción de la resolución espacial a través de diversas operaciones como podrían ser el *max-pooling* o el uso de capas convolucionales, obteniendo así como resultado unas capas de características de baja resolución pero semánticamente ricas.

2. Decodificación por decoder: El decodificador opera con las características comprimidas del codificador para reconstruir la predicción final. Esto lo hará mediante la utilización de operaciones de sobremuestreo, a menudo implementadas mediante *upsampling* [3, 37] o mediante el uso de índices de *max-pooling* [45, 74], agrandando la entrada para extraer características de alta resolución [45, 34, 3].
3. Salida y Optimización: La salida final es un mapa de segmentación que se alimenta a una capa *Softmax* para asignar probabilidades de clasificación a cada píxel de forma independiente [34, 4, 45], generando una predicción densa [45, 4].

Kernel

En el ámbito de las redes neuronales, un *kernel* es un conjunto de filtros con pesos aprendibles que constituyen el componente fundamental de las capas convolucionales para convertir datos de entrada en mapas de características [34]. Operativamente, el *kernel* se desliza sobre la entrada utilizando un *stride*, realizando una operación matemática que produce una salida espacial encargada de capturar jerarquías de información, desde bordes simples hasta formas complejas [28, 34, 38]. Esta estrategia es intrínseca al procesamiento visual, permitiendo que las computaciones se compartan y que el sistema sea eficiente al manejar imágenes de alta resolución [38].

El diseño del *kernel*, es una decisión arquitectónica crítica que determina el campo receptivo; por ejemplo, mientras que los *kernels* de 3×3 son el estándar por su eficiencia en *hardware*, modelos modernos han demostrado que los *kernels* de 7×7 pueden capturar un contexto global mucho más amplio [38].

Además de la extracción de características, el concepto de *kernel* se extiende a funciones matemáticas de comparación, como el *Central Kernel Alignment (CKA)*, que utiliza un *kernel* lineal para medir la similitud de representación entre los espacios de embebido latente de diferentes arquitecturas [31].

Stride

El *stride* es un parámetro crítico que define la magnitud del desplazamiento de un *kernel* a medida que recorre los datos de entrada en una red neuronal [34, 14]. En las capas del codificador, el uso de un *stride* mayor a uno constituye una técnica de *downsampling* que reduce progresivamente la resolución espacial de los mapas de características [3, 53]. Esta operación

permite que los píxeles resultantes abarquen un contexto de imagen de entrada más amplio, facilitando la extracción de características semánticas de alto nivel y mejorando la invarianza traslacional del modelo [34, 3, 38].

En el decodificador, el *stride* es esencial para las operaciones de *upsampling*, donde se utiliza para expandir representaciones comprimidas hacia mapas de alta resolución [34, 3]. El tamaño final de la salida depende de la interacción matemática entre las dimensiones de entrada, el *padding*, el tamaño del *kernel* y el valor del *stride* aplicado [34, 65]. La gestión precisa de este paso es vital para reconstruir con fidelidad la información espacial y asegurar una delineación precisa de los límites de los objetos en tareas de segmentación densa [53, 29, 4].

Transformer

La arquitectura del *Transformer* es un modelo de procesamiento de secuencias basado íntegramente en mecanismos de autoatención, eliminando la necesidad de recurrencia o convoluciones para capturar dependencias [69, 35]. A diferencia de las redes recurrentes que procesan elementos de forma secuencial, el *Transformer* permite el procesamiento en paralelo de toda la secuencia, lo que facilita el modelado de relaciones de largo alcance y mejora significativamente la escalabilidad en grandes conjuntos de datos [35, 27].

Su estructura estándar se compone de un codificador y un decodificador que apilan múltiples bloques idénticos, cada uno equipado con capas de atención de múltiples cabezales y redes neuronales [69, 4].

Mecanismos residuales

Los mecanismos residuales introducen conexiones aditivas, conocidas como *skip connections*, que permiten que la señal de entrada de un bloque se sume directamente a su salida. De este modo, el bloque no aprende una transformación completa, sino una corrección respecto a la señal original, formalizando la salida como la suma entre la entrada y una función residual aprendida [28]. Este planteamiento facilita que la información fluya sin alteraciones innecesarias a través de la red y reduce la degradación del rendimiento a medida que aumenta la profundidad.

Desde el punto de vista del entrenamiento, estas conexiones mejoran la propagación de gradientes durante la retropropagación y estabilizan el proceso de optimización, lo que permite entrenar redes considerablemente más profundas. Cuando las dimensiones de entrada y salida no coinciden, la conexión residual se implementa mediante proyecciones lineales; en otros

casos, se emplean saltos de identidad directos que preservan completamente la señal original. En conjunto, estos mecanismos mantienen rutas de transmisión eficientes para la información y los gradientes, facilitando el aprendizaje de funciones complejas en arquitecturas profundas [60].

Skip Connections

Conexiones encargadas de mitigar la pérdida de información espacial que ocurre durante el *downsampling* en el codificador [4]. Transfieren características de alta resolución desde las capas tempranas del codificador a las capas correspondientes del decodificador, fusionando las características codificadas con los detalles espaciales finos, siendo estos los límites y contornos, del inicio de la red [3, 53].

Para lograr esto, se emplea una capa de concatenación que fusiona la salida de las capas convolucionales del *encoder* con las capas de deconvolución del *decoder* [53, 34].

Upsampling

El *upsampling*, también denominado deconvolución, es la función principal del decodificador, definida como la operación de mapeo que transforma representaciones de características comprimidas de baja resolución en mapas de características de alta resolución espacial para la clasificación a nivel de píxel [3, 34].

Esto puede implementarse mediante interpolación, convolución transpuesta o decodificadores MLP [34, 6]. La implementación también puede optar por utilizar métodos sin aprendizaje, aunque esto puede resultar en un rendimiento inferior en comparación con los métodos que sí aprenden del entorno [3].

Para preservar los contornos finos y garantizar la localización precisa, el decodificador se basa en la fusión de características de niveles previos del codificador mediante *skip connections* [53, 3, 4].

Downsampling

El *downsampling* es una técnica fundamental utilizada en las redes neuronales convolucionales y en el *encoder* de las arquitecturas de codificador-decodificador para transformar la información de entrada en una representación de características reducida de alto nivel [34, 3].

El propósito central de esta operación es doble: por un lado, aliviar la carga computacional y prevenir el *overfitting*; por otro lado, lograr una mayor *invarianza traslacional* sobre pequeños desplazamientos espaciales en la imagen de entrada, lo cual robustece la clasificación. Esto se realiza a costa de una pérdida progresiva de la resolución espacial en los mapas de características, lo cual reduce el detalle de los límites, un factor que debe compensarse en la fase de decodificación [3, 53, 4].

El mecanismo principal para lograr el submuestreo es la operación de *max-pooling*, que es el núcleo del *downsampling* y se utiliza para disminuir el tamaño del mapa de características [34, 53]. Lo que hace para llevarlo a cabo es dividir dicho mapa de entrada en pequeñas regiones o ventanas. Tras ello, extrae el valor máximo dentro de una de las ventanas, utilizándolo como el único valor representativo para esa área en el nuevo mapa de características reducido. Finalmente, la ventana se desliza a lo largo de la entrada con un *stride* predefinido, permitiendo que cada píxel resultante abarque un contexto de imagen de entrada más amplio a medida que se reduce la resolución [74, 34, 3].

Aunque las sucesivas capas de *max-pooling* y submuestreo pueden lograr una mayor invarianza para una clasificación robusta, la contraparte es una pérdida progresiva de la resolución espacial en los mapas de características. Esta pérdida resulta en una representación cada vez más inexacta en cuanto al detalle de los límites de los objetos, lo cual es perjudicial para tareas de predicción densa, como la segmentación, donde la delineación precisa es vital [3, 4]. Sin embargo, el *downsampling* tiene un beneficio inherente: logra que cada píxel en el mapa de características posterior abarque un contexto de imagen de entrada más grande.

Vision Transformer (ViT)

La arquitectura del *Vision Transformer (ViT)* se erige como el paradigma dominante después de las CNN en tareas de reconocimiento visual [38, 35, 4]. Opera al inicio con una fase de tokenización no superpuesta, donde la imagen de entrada es segmentada en parches, aplanados, y proyectados linealmente mediante una transformación a un espacio embebido [4]. A estos *embeddings* se les adjuntan codificaciones posicionales para conservar la información espacial [4].

Esta secuencia de *tokens* enriquecida se introduce en un *encoder* de *Transformer* que consiste en múltiples bloques idénticos, equipados con el mecanismo de *Multi-Head Self-Attention*. El *self-attention* permite que cada *token* capture relaciones de largo alcance y el contexto global con todos

los demás *tokens*, facilitando una interpretación más inclusiva de patrones intrincados en los datos [35, 4].

En tareas de segmentación semántica, los modelos basados en ViT se emplean habitualmente como extractores de características globales. El *encoder* procesa la imagen y genera representaciones con un alto contenido semántico, pero con una resolución espacial reducida. Para obtener un mapa de segmentación a nivel de píxel, estas representaciones se combinan con un decodificador que permite recuperar la resolución espacial y producir la salida final [4].

La salida final se genera a través de una capa convolucional seguida de una activación *Softmax*, proporcionando un mapa de probabilidades detallado para la clasificación a nivel de píxel [74, 4].

No linealidades y capa de decisión

En este apartado se describen los mecanismos que permiten a una red neuronal pasar de combinar características de forma meramente lineal a tomar decisiones discriminativas útiles para la tarea de segmentación. Por un lado, las no linealidades introducidas por las funciones de activación determinan la capacidad expresiva del modelo y condicionan la estabilidad del entrenamiento, al regular cómo se propagan las señales y los gradientes a través de capas profundas. Por otro lado, la capa de decisión final transforma las salidas internas del modelo en predicciones interpretables, típicamente en forma de distribuciones de probabilidad por clase, lo que habilita la asignación coherente de etiquetas a cada píxel en escenarios de predicción densa [21].

Función de activación

Las funciones de activación son componentes esenciales de las redes neuronales, ya que introducen no linealidades tras las operaciones lineales de convolución o proyección. Gracias a estas no linealidades, la red es capaz de aproximar funciones complejas y aprender representaciones jerárquicas que capturan patrones visuales de distinta escala y complejidad [21].

Sin la aplicación de funciones de activación, la composición de múltiples capas lineales se reduciría a una única transformación lineal, limitando severamente la capacidad expresiva del modelo. Para prácticas de segmentación semántica, estas funciones permiten discriminar estructuras complejas y no lineales, como bordes irregulares o sendas difusas, contribuyendo a la coherencia espacial de las predicciones generadas [45].

Unidad Lineal Rectificada (ReLU)

La Unidad Lineal Rectificada (*ReLU*) es una función de activación no lineal definida por una relación simple que conserva los valores positivos de la entrada y anula los negativos:

$$\text{ReLU}(x) = \max(0, x) \quad (3.1)$$

Su simplicidad computacional y su comportamiento no saturante han favorecido su adopción generalizada en arquitecturas profundas modernas [34].

Desde el punto de vista del entrenamiento, *ReLU* contribuye a mitigar el problema del desvanecimiento del gradiente, facilitando una propagación más estable de la información durante la retropropagación. En arquitecturas de segmentación como U-Net o SegNet, se emplea de forma sistemática tras las convoluciones para introducir no linealidad sin comprometer la eficiencia, permitiendo el aprendizaje efectivo de patrones espaciales complejos [38, 43].

Función Softmax

La función *Softmax* se emplea habitualmente en la capa final de redes neuronales de clasificación y segmentación para transformar las salidas no normalizadas del modelo, conocidas como *logits*, en una distribución de probabilidad:

$$\text{Softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K \quad (3.2)$$

Esta transformación garantiza que los valores resultantes sean no negativos y que su suma sea igual a uno, lo que permite interpretarlos directamente como probabilidades asociadas a cada clase [21].

En el contexto de la segmentación semántica, la función *Softmax* se aplica de forma independiente a cada píxel, produciendo para cada uno de ellos un vector de probabilidades sobre las clases posibles. La etiqueta final asignada corresponde a la clase con mayor probabilidad, lo que da lugar a mapas de segmentación interpretables y coherentes. Este mecanismo resulta especialmente adecuado en problemas multiclase, donde es necesario modelar explícitamente la competencia entre clases en cada posición espacial de la imagen [53, 3].

Consideraciones de generalización y propiedades espaciales

En esta subsección se agrupan conceptos que determinan hasta qué punto un modelo de segmentación es fiable fuera del conjunto de entrenamiento y cómo conserva la estructura geométrica de la escena al producir predicciones densas. Por un lado, se revisa el *overfitting* como riesgo central de generalización en aprendizaje profundo y las estrategias habituales para controlarlo durante el entrenamiento. Por otro lado, se introducen propiedades espaciales clave en visión por computador, en particular la invarianza y la equivarianza traslacional, que describen cómo debe comportarse el modelo ante desplazamientos en la imagen y explican por qué ciertas arquitecturas resultan especialmente adecuadas para mantener coherencia espacial en tareas de segmentación semántica [21, 14, 35].

Invarianza traslacional

La invarianza traslacional describe la capacidad de un modelo de visión por computador para reconocer una misma característica u objeto aunque aparezca desplazado a distintas posiciones dentro de la imagen. En el contexto de las redes neuronales convolucionales, esta propiedad permite que ciertos patrones visuales como bordes, texturas o estructuras se identifiquen de forma consistente con independencia de su localización exacta en la escena [34].

Esta propiedad no surge de manera estricta en una sola capa, sino que se construye progresivamente a través de la combinación de convoluciones locales y operaciones de submuestreo mediante *pooling* o *downsampling* [14]. Las capas convolucionales comparten pesos en toda la imagen, lo que hace que un mismo filtro responda de igual manera ante un patrón dado, esté donde esté. A medida que se incrementa el campo receptivo en capas más profundas, la red integra información de regiones cada vez más amplias, reduciendo la sensibilidad a pequeños desplazamientos espaciales. En aplicaciones de clasificación y segmentación semántica, esta propiedad es clave para garantizar que la predicción asociada a un objeto o región no dependa de su posición absoluta dentro de la imagen [3].

Equivarianza traslacional

La equivarianza traslacional describe la propiedad por la cual una transformación aplicada a la entrada de un modelo se refleja de forma coherente en su salida. En el caso de traslaciones espaciales, esto implica que si un objeto se desplaza dentro de la imagen de entrada, la representación o predic-

ción generada por el modelo se desplaza en la misma dirección y magnitud, preservando la correspondencia espacial entre entrada y salida [35].

En redes neuronales convolucionales, esta propiedad se deriva directamente de la operación de convolución y de la compartición de pesos, que garantizan que un mismo patrón visual active respuestas equivalentes en distintas posiciones de la imagen. En tareas de segmentación semántica, la equivarianza traslacional es esencial, ya que el objetivo no es únicamente reconocer la presencia de una clase, sino asignar una etiqueta correcta a cada píxel. Gracias a esta propiedad, los mapas de segmentación mantienen una alineación precisa con la geometría de la escena, permitiendo que las estructuras detectadas conserven su forma y localización relativas respecto a la imagen original [38].

En contraste con la invarianza traslacional, que busca reducir la sensibilidad del modelo a la posición absoluta de los objetos para favorecer el reconocimiento global, la equivarianza preserva explícitamente la información espacial. Ambas propiedades no son excluyentes, sino complementarias: mientras la invarianza resulta clave en tareas de clasificación, la equivarianza constituye un requisito fundamental en problemas de predicción densa, como la segmentación semántica, donde la coherencia espacial de la salida es crítica.

Arquitecturas de Segmentación Semántica

En esta sección se presentan las principales arquitecturas de segmentación semántica empleadas en el ámbito de la visión por computador, las cuales materializan los principios teóricos descritos anteriormente mediante diseños de red específicos orientados a la predicción densa. Se analizan arquitecturas representativas basadas en esquemas de codificador-decodificador y en la agregación jerárquica de características, como *U-Net*, *Feature Pyramid Network* y *UPerNet*, así como enfoques más recientes que incorporan mecanismos de atención global mediante *Transformers*, como *SegFormer*. Asimismo, se incluye *PSPNet*, una arquitectura pionera en la integración explícita de contexto multi-escala a través de pirámides de agrupamiento. Cada una de estas arquitecturas introduce estrategias diferenciadas para capturar información semántica y preservar la coherencia espacial, constituyendo referentes fundamentales en el desarrollo y la evolución de los modelos de segmentación semántica.

U-Net

La red neuronal convolucional *U-Net* constituye un paradigma fundamental en el ámbito de la segmentación densa debido a su capacidad para operar eficientemente con conjuntos de datos de entrenamiento limitados [53]. Esta arquitectura presenta una topología simétrica en forma de U que integra un codificador y un decodificador [53, 13]. El codificador aplica sucesivamente convoluciones seguidas de unidades de activación lineal rectificada y operaciones de *max-pooling* para la extracción de características contextuales y la reducción de la dimensionalidad espacial [53, 34]. Por su parte, el decodificador realiza procesos de *upsampling* con el fin de recuperar la resolución original de la imagen [53, 3].

El componente diferencial de este modelo reside en la implementación de conexiones de salto que concatenan mapas de características de alta resolución del codificador con las capas correspondientes del decodificador [53, 15]. Dicho mecanismo permite la preservación de detalles espaciales que se degradan habitualmente durante el submuestreo, garantizando una localización precisa de los límites de los objetos en tareas críticas como la imagen médica o la teledetección [13].

Autoencoder

Un *autoencoder* es una arquitectura de red neuronal no supervisada cuyo objetivo es aprender una representación compacta de los datos de entrada sin utilizar etiquetas. La red se compone de dos partes claramente diferenciadas: un codificador, que transforma la entrada original de alta dimensionalidad en una representación latente de menor dimensión, y un decodificador, que reconstruye la entrada a partir de dicha representación minimizando el error de reconstrucción. Este esquema de compresión y reconstrucción obliga al modelo a retener la información más relevante de los datos, eliminando redundancias y atenuando el ruido [45].

Desde un punto de vista práctico, el espacio latente aprendido por el *autoencoder* recoge patrones estructurales y características salientes de los datos, lo que lo hace útil como etapa previa de extracción de características. En tareas de segmentación semántica, estas representaciones pueden emplearse para inicializar modelos más complejos o para aprender descriptores compactos que faciliten la posterior clasificación píxel a píxel [21].

Existen extensiones del *autoencoder* clásico que refuerzan esta capacidad. Los *Denoising Autoencoders* entrenan el modelo a partir de entradas degradadas, forzándolo a recuperar la señal original y mejorando así la

robustez frente a perturbaciones. Por su parte, los *autoencoders* variacionales introducen una formulación probabilística del espacio latente, lo que permite modelar la distribución de los datos y generar nuevas muestras coherentes. En todos los casos, el principio fundamental es el mismo: aprender una representación latente informativa que facilite tareas posteriores de análisis o decisión [45, 31].

Evaluación experimental de la segmentación semántica

En esta subsección se formaliza la evaluación experimental de un sistema de segmentación semántica, estableciendo un marco común para medir su rendimiento de manera reproducible y comparable [41, 64].

Para ello, se definen las métricas de evaluación empleadas, combinando medidas de solapamiento, rendimiento temporal y propiedades geométricas derivadas de la máscara con el objetivo de caracterizar tanto la equivalencia entre segmentaciones como su impacto estructural sobre las trazas detectadas [20, 75].

Métricas de evaluación

Para analizar el comportamiento de un modelo de segmentación semántica no basta con observar visualmente las máscaras generadas, sino que resulta necesario cuantificar de forma objetiva sus aciertos y errores, especialmente cuando la clase de interés corresponde a estructuras finas, discontinuas o de bajo contraste, como las trazas de herbivoría.

En este contexto, resulta habitual combinar métricas de solapamiento, que miden la coincidencia entre máscaras binarias, con medidas derivadas de la geometría de la predicción.

Adicionalmente, la evaluación puede formularse en términos de la matriz de confusión, donde la clase positiva corresponde a píxeles pertenecientes a la traza de herbivoría, mientras que la clase negativa representa el suelo sin traza.

En la comparación realizada entre rutas de inferencia, las métricas de solapamiento se calculan entre la máscara obtenida al cargar el modelo mediante `pickle` y la adquirida al ejecutar la versión exportada a *TorchScript*, sin emplear anotación externa [48].

Intersección sobre la Unión (IoU)

La métrica de Intersección sobre la Unión (*Intersection over Union*, IoU) mide el solapamiento relativo entre dos máscaras binarias. En esta comparativa se define como:

$$\text{IoU} = \frac{|M_{\text{pickle}} \cap M_{\text{torch}}|}{|M_{\text{pickle}} \cup M_{\text{torch}}|} \quad (3.3)$$

donde M_{pickle} es la máscara obtenida por la ruta de inferencia que carga el modelo mediante `pickle` y M_{torch} la máscara generada por la ruta *PyTorch/TorchScript*. Un valor cercano a uno indica una equivalencia espacial alta, mientras que valores menores reflejan discrepancias localizadas [41, 13, 64].

Desde un punto de vista geométrico, esta comparación puede interpretarse como el solapamiento entre ambas máscaras, tal y como se ilustra en la Figura 3.2.

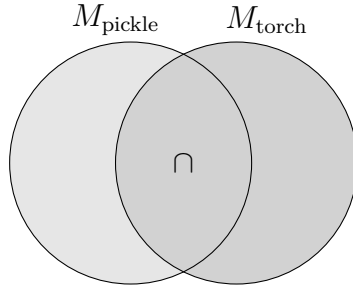


Figura 3.2: Interpretación geométrica de IoU como solapamiento entre la máscara obtenida mediante la ruta de carga con `pickle` y la máscara obtenida mediante la ruta *TorchScript*.

Coficiente Dice

El coeficiente *Dice* es otra medida de solapamiento, especialmente utilizada en segmentación binaria, y en este contexto se calcula como:

$$\text{Dice} = \frac{2 |M_{\text{pickle}} \cap M_{\text{torch}}|}{|M_{\text{pickle}}| + |M_{\text{torch}}|} \quad (3.4)$$

Al igual que IoU, valores altos implican que ambas rutas de inferencia generan máscaras muy similares; no obstante, *Dice* tiende a ser menos severo que IoU en casos de clases minoritarias [64, 41].

Métricas temporales de rendimiento

Para evaluar la eficiencia práctica de cada enfoque se midieron tiempos de carga y tiempos de inferencia por *fold* en el banco de pruebas. En particular:

- Tiempo de carga (**load_ms**): Tiempo necesario para cargar el modelo desde disco.
- Tiempo de inferencia medio (**infer_ms_mean**): Media del tiempo de ejecución del *forward* sobre una misma imagen, tras un calentamiento para estabilizar el rendimiento.
- Desviación estándar (**infer_ms_std**): Variabilidad del tiempo de inferencia entre repeticiones, empleado para detectar fluctuaciones del sistema.

Métricas geométricas derivadas de la máscara

Además del solapamiento entre predicciones, se calcularon métricas simples sobre la máscara binaria para interpretar cuantitativamente la estructura de las trazas detectadas:

- Área segmentada (**area_px**): Número de píxeles clasificados como traza, interpretado como la extensión total detectada.
- Longitud del esqueleto (**skeleton_len_px**): Longitud en píxeles del esqueleto de la máscara, utilizada como aproximación a la longitud de las sendas detectadas [75].
- Número de componentes conexas (**n_components**): Cantidad de regiones desconectadas segmentadas, indicador de discontinuidades en la detección [20].
- Grosor medio estimado (**mean_thickness_px**): Aproximación del grosor medio de la traza, calculada a partir de la relación entre área y longitud del esqueleto [75].

Estas medidas complementan las métricas de solapamiento, ya que permiten comprobar si pequeñas discrepancias entre rutas de inferencia se traducen en cambios apreciables en la morfología global de las trazas.

4. Técnicas y herramientas

Este capítulo tiene como finalidad describir, de manera ordenada y argumentada, las técnicas metodológicas y las herramientas empleadas a lo largo del desarrollo del proyecto.

4.1. Técnicas metodológicas

A lo largo del desarrollo se han aplicado técnicas orientadas a organizar el trabajo, mantener consistencia en el código y facilitar la evolución incremental del prototipo. En concreto, se ha adoptado una metodología de planificación inspirada en *Scrum*, se han seguido convenciones de estilo para mejorar la legibilidad y mantenibilidad del código, y se ha aplicado un enfoque sistemático de construcción de interfaz mediante un sistema de utilidades, con el fin de asegurar coherencia visual sin introducir deuda técnica innecesaria.

Planificación iterativa basada en Scrum

La planificación se ha estructurado en iteraciones, denominadas *sprints*, con el objetivo de descomponer el alcance del proyecto en tareas abordables y reducir el riesgo de bloqueo en fases finales. Para cada iteración se han definido objetivos concretos, priorizados en función del valor aportado al sistema y de sus dependencias técnicas, manteniendo una visión continua del trabajo pendiente mediante un *backlog*. Este enfoque ha permitido combinar desarrollo funcional y redacción de memoria de forma coordinada, incorporando revisiones periódicas del avance y ajustes razonables cuando se detectaban desajustes entre estimación y esfuerzo real.

Convenciones de estilo y guía PEP8

Para asegurar consistencia y facilitar la revisión del código, se han seguido convenciones de estilo alineadas con PEP8 en los módulos Python del proyecto. Esta decisión contribuye a que el código sea más homogéneo, reduce fricción al introducir nuevas funcionalidades y mejora la mantenibilidad del repositorio, especialmente en zonas críticas como la gestión de rutas, la persistencia de resultados y la integración del *pipeline* de inferencia. Además, favorece la detección temprana de errores comunes y simplifica la colaboración y supervisión del proyecto por parte de terceros.

Diseño de interfaz con Tailwind CSS y DaisyUI

El diseño de la interfaz se ha abordado mediante un enfoque basado en utilidades con *Tailwind CSS*, complementado con *DaisyUI* como capa de componentes. Este planteamiento permite construir vistas consistentes a partir de clases de bajo nivel, manteniendo un control fino sobre espaciados, tipografía y jerarquía visual, y evitando el coste de sobrescrituras frecuentes asociado a estilos rígidos. En la práctica, el uso de un sistema de utilidades favorece la escalabilidad del *frontend*, ya que la coherencia visual emerge de patrones reutilizables y de una configuración centralizada, lo que resulta especialmente útil en una aplicación con pantallas heterogéneas, como procesamiento de imágenes, visor cartográfico y paneles de administración.

Internacionalización e independencia del idioma

El sistema se ha diseñado con capacidad de internacionalización, desacoplando los textos visibles del idioma concreto mediante el uso de catálogos de traducción. Esta técnica permite que la aplicación sea neutral desde el punto de vista lingüístico, evitando duplicidad de código y facilitando añadir nuevos idiomas sin modificar la lógica principal. En el caso del contenido dinámico generado en cliente, se sigue una estrategia consistente para que los textos sigan siendo localizables y no queden incrustados de manera rígida en el *frontend*.

Gestión de dependencias, reproducibilidad y calidad

Para garantizar reproducibilidad, el entorno de ejecución se define mediante un fichero de dependencias versionadas y un entorno aislado de Python. Esta decisión reduce problemas derivados de incompatibilidades entre versiones, facilita replicar el entorno de desarrollo y estabiliza el despliegue. En

paralelo, se siguen criterios de estilo y organización del código, manteniendo convenciones consistentes y promoviendo cambios incrementales. Finalmente, el proyecto incorpora una estrategia de pruebas automatizadas que permite detectar regresiones en funcionalidades críticas, especialmente en aquellas que integran inferencia, persistencia y comunicación cliente–servidor.

4.2. Herramientas y tecnologías empleadas

En esta sección se describen las herramientas principales utilizadas para implementar el sistema, agrupadas según su función dentro del *stack*. La finalidad no es enumerar exhaustivamente cada dependencia, sino destacar aquellas que han sido determinantes para construir la aplicación, mantener un desarrollo sostenible y asegurar reproducibilidad.

Entorno de ejecución y dependencias

- **Python:** lenguaje base del proyecto.
- **pip y requirements.txt:** instalación y fijación de dependencias para reproducir el entorno.
- **Entorno virtual y conda:** mecanismos de aislamiento del entorno, empleados para evitar colisiones de dependencias y estabilizar la ejecución.

Backend y capa web

- **Flask:** *microframework* utilizado para implementar el servidor, rutas y lógica de aplicación.
- **Jinja2:** motor de plantillas para renderizado del HTML en servidor.
- **Werkzeug:** utilidades WSGI y soporte de infraestructura del ecosistema Flask.
- **Flask-Login:** gestión de autenticación, sesión y permisos por rol.
- **Flask-WTF y WTForms:** formularios y validación robusta de entradas.
- **Flask-Babel y Babel:** internacionalización y localización de la interfaz.

- **Flask-Admin:** soporte para vistas y operaciones de administración.

Persistencia y acceso a datos

- **SQLite:** base de datos embebida utilizada para persistencia durante el desarrollo.
- **SQLAlchemy y Flask-SQLAlchemy:** capa ORM para desacoplar el acceso a datos y facilitar mantenibilidad.

Inferencia, modelos y procesamiento de imagen

- **PyTorch:** marco principal para ejecutar el modelo de segmentación en inferencia.
- **pickle:** mecanismo de deserialización para modelos heredados.
- **segmentation-models-pytorch:** soporte para arquitecturas y componentes de segmentación.
- **timm:** catálogo de *backbones* y modelos preentrenados empleados como codificadores.
- **safetensors:** formato seguro y eficiente para el almacenamiento y carga de pesos de modelos cuando procede.
- **Jupyter Notebook:** entorno utilizado para explorar el *pipeline* original, replicar inferencias y validar resultados antes de integrarlos en la aplicación.
- **NumPy y Pillow:** utilidades para manipulación de arrays y lectura o conversión de imágenes.

Frontend, estilos y experiencia de usuario

- **HTML5, CSS y JavaScript:** tecnologías base del *frontend*.
- **Tailwind CSS:** *framework* de utilidades empleado para el diseño y la composición visual.
- **DaisyUI:** extensión basada en componentes sobre Tailwind, utilizada para acelerar la construcción de elementos recurrentes manteniendo coherencia.

Cartografía y datos geoespaciales

- **Leaflet:** biblioteca de mapas en cliente utilizada para el visor interactivo.
- **OpenStreetMap:** mapa base de referencia para navegación e interacción.
- **PNOA mediante servicios WMS:** fuente de ortofotografía de alta resolución utilizada para visualización y obtención de recortes.

Pruebas, diagramación y documentación

- **pytest:** *framework* de pruebas automatizadas para reducir regresiones.
- **Selenium:** herramienta utilizada para automatizar pruebas funcionales en un navegador real, integrada con **pytest** para validar los flujos principales de usuario sobre la interfaz web.
- **SonarQube:** herramienta de análisis estático empleada para revisar la calidad del código, detectar problemas de mantenibilidad, duplicidad, complejidad, posibles errores y vulnerabilidades.
- **Git y GitHub:** control de versiones y gestión del repositorio.
- **Visual Studio Code:** editor principal de desarrollo.
- **Diagrams.net (Draw.io):** herramienta empleada para la elaboración de diagramas de arquitectura y flujo.
- **L^AT_EX, T_EXStudio y MiK_TE_X Console:** herramientas utilizadas para redactar, compilar y gestionar la memoria y sus dependencias tipográficas.

Despliegue y ejecución en servidor

- **Gunicorn:** servidor WSGI utilizado para ejecutar la aplicación Flask en el servidor de la Universidad de Burgos.
- **Docker:** herramienta empleada como apoyo al despliegue y a la reproducibilidad del entornos.

Gestión del proyecto

- **Zube.io:** herramienta utilizada para gestionar el *backlog* y organizar las tareas por *sprints*, facilitando el seguimiento del avance y la priorización del trabajo.

Comparativa entre Bootstrap y Tailwind CSS

En el diseño e implementación de la interfaz web del proyecto resulta necesario seleccionar un *framework* de estilos que aporte productividad sin comprometer la mantenibilidad ni la calidad del código. Entre las alternativas más consolidadas se encuentran Bootstrap, orientado a componentes predefinidos, y Tailwind CSS, basado en un paradigma de utilidades. A continuación se presenta una comparativa crítica entre ambas opciones, justificando la elección final de Tailwind CSS para el desarrollo de esta página.

Bootstrap propone un conjunto amplio de componentes listos para usar, tales como barras de navegación, tarjetas o sistemas de rejilla, acompañados de un diseño visual relativamente homogéneo. Esta aproximación permite, en un primer momento, acelerar el desarrollo, ya que se pueden componer interfaces funcionales mediante la combinación de bloques estándar. Además, Bootstrap ofrece una documentación madura y una comunidad extensa, lo que facilita la resolución de dudas y la búsqueda de ejemplos; sin embargo, esta misma estandarización introduce ciertas limitaciones. Por un lado, la personalización profunda del diseño suele requerir sobrescribir reglas de estilo, lo que incrementa la complejidad de las hojas CSS y genera deuda técnica con facilidad. Por otro lado, las interfaces tienden a parecerse entre sí, de forma que la identidad visual del proyecto puede quedar diluida si no se realiza un esfuerzo considerable de adaptación.

En contraste, Tailwind CSS se basa en clases utilitarias de bajo nivel que describen propiedades concretas, tales como márgenes, tipografía o colores, aplicadas directamente sobre el marcado HTML. Esta filosofía permite construir componentes a medida a partir de combinaciones de utilidades, sin depender de un conjunto rígido de componentes predefinidos. Como consecuencia, el diseño resultante se ajusta mejor a los requisitos específicos del proyecto y favorece la creación de un sistema de diseño propio, más alineado con la identidad visual deseada. Además, Tailwind CSS integra mecanismos de purga de clases no utilizadas, lo que reduce de manera significativa el tamaño del fichero CSS final y mejora el rendimiento en producción.

No obstante, Tailwind CSS presenta también ciertos inconvenientes. Al inicio, la curva de aprendizaje puede parecer más pronunciada, ya que obliga a pensar en términos de propiedades de bajo nivel y no tanto en componentes cerrados. Asimismo, la presencia de múltiples clases en cada elemento puede dar lugar a un marcado aparentemente más denso, pero, cuando se interioriza el conjunto de utilidades y se adoptan buenas prácticas, como la extracción de componentes reutilizables y el uso de configuración centralizada, el resultado es un código más predecible y coherente, con menos necesidad de sobrescrituras complejas.

Desde una perspectiva de la ingeniería del software, la elección entre ambos enfoques afecta de forma directa a la mantenibilidad y escalabilidad del proyecto. Bootstrap ofrece rapidez inicial, pero puede derivar en hojas de estilo difíciles de gestionar cuando se requiere un alto grado de personalización. En cambio, Tailwind CSS promueve una relación más estrecha entre el diseño y el código, ya que el sistema de diseño se expresa mediante utilidades consistentes que se reutilizan en toda la aplicación. Esto reduce la duplicidad de estilos, minimiza el riesgo de efectos colaterales al modificar clases existentes y facilita la evolución del producto a largo plazo.

Finalmente, he optado por Tailwind CSS debido a su capacidad para generar interfaces más ligeras, a su enfoque modular y a su mejor integración con patrones modernos de desarrollo basado en componentes. Gracias a su sistema de utilidades, ha sido posible definir un diseño adaptado a las necesidades concretas del proyecto, mantener un control fino sobre el *bundle* de estilos y reducir la deuda técnica asociada a la personalización del *frontend*. En definitiva, Tailwind CSS proporciona un equilibrio más adecuado entre flexibilidad, rendimiento y mantenibilidad, lo que lo convierte en la opción más idónea para este desarrollo.

Comparativa de motores y fuentes de datos geoespaciales

La elección del motor geoespacial y de la fuente de datos constituye una decisión crítica en el diseño de aplicaciones web orientadas al análisis terrenal y a la detección de elementos lineales, ya que condiciona de forma directa tanto el nivel de detalle alcanzable como la escalabilidad, la reproducibilidad y la viabilidad legal del sistema. En este contexto, se han analizado cinco alternativas representativas del estado del arte actual, valorando sus capacidades técnicas, sus limitaciones y su adecuación a los objetivos del proyecto.

En primer lugar, Google Earth Engine se presenta como una plataforma de computación geoespacial en la nube que destaca por su amplio catálogo de datos científicos y por su capacidad de procesado a gran escala [23, 26, 22]. Permite trabajar de forma eficiente con series temporales extensas y con colecciones como Sentinel-2 o Landsat, facilitando la generación de mosaicos, máscaras de nubes o índices espectrales. No obstante, su principal limitación en este proyecto radica en que las fuentes satelitales abiertas disponibles habitualmente en estos catálogos no alcanzan la resolución necesaria para detectar trazas finas, como senderos o caminos estrechos. Además, conviene subrayar que la imagerie de alta resolución visible en Google Earth o Google Maps no forma parte de un repositorio libre de datos exportables para análisis automático. Los términos asociados a Google Maps Platform restringen expresamente la extracción, descarga masiva, almacenamiento y creación de contenido derivado a partir de Google Maps Content, lo que impide utilizar capturas o teselas de mapas base de Google como entrada de un *pipeline* de segmentación semántica reproducible y conforme a licencia [25].

Como alternativa orientada a la integración en aplicaciones web, Sentinel Hub y el Copernicus Data Space ofrecen interfaces de programación que permiten solicitar imágenes procesadas bajo demanda. Esta aproximación resulta especialmente cómoda en arquitecturas cliente-servidor, ya que simplifica el flujo de petición y respuesta mediante recortes espaciales y temporales. Sin embargo, al igual que en el caso anterior, la resolución de las imágenes Sentinel constituye un factor limitante cuando se requiere un nivel de detalle submétrico, lo que reduce su idoneidad para el objetivo concreto del proyecto.

Una tercera opción, de especial relevancia en el ámbito nacional, es el uso de ortofotos aéreas de alta resolución, destacando en España el Plan Nacional de Ortofotografía Aérea, PNOA. Este conjunto de datos proporciona imágenes con resoluciones del orden de decímetros, muy superiores a las de los sensores satelitales de libre acceso. Además, su disponibilidad a través de servicios WMS y WMTS, junto con opciones de descarga directa desde el Centro de Descargas del CNIG, lo convierte en una solución sólida tanto para la visualización como para el análisis reproducible [32, 9]. Desde una perspectiva académica, el uso de fuentes oficiales y abiertas refuerza la validez metodológica del trabajo y evita problemas derivados de licencias restrictivas.

En contraposición, el uso directo de imagerie procedente de Google Maps Imagery o Google Earth como fuente de datos para análisis automático

presenta importantes inconvenientes. A pesar de su elevada calidad visual, estas plataformas imponen restricciones estrictas en cuanto a la extracción, almacenamiento, descarga masiva y procesamiento de imágenes, lo que dificulta su empleo como motor principal en un proyecto de carácter académico y analítico [25]. Por este motivo, su utilización queda prácticamente relegada a contextos de visualización, no siendo recomendable como base para flujos de detección y análisis a escala.

Finalmente, en lo relativo a la capa de visualización, se ha considerado el uso de Folium frente a la integración directa de bibliotecas JavaScript como Leaflet. Aunque Folium resulta adecuado para prototipos rápidos generados desde Python, su naturaleza estática y su dependencia del backend lo hacen menos apropiado para aplicaciones web interactivas complejas. En consecuencia, una solución basada en Leaflet u OpenLayers en el frontend, combinada con un backend que sirva datos vectoriales en formato JSON, ofrece mayor flexibilidad y control [1, 47].

A la luz de este análisis comparativo, la solución adoptada en el proyecto consiste en utilizar ortofotografía aérea de alta resolución, concretamente PNOA, como fuente principal de datos geoespaciales, integrada sobre un visor web construido con Leaflet y apoyado en OpenStreetMap como mapa base para la interacción del usuario. Esta elección garantiza el nivel de detalle necesario para la detección de trazas finas, asegura la reproducibilidad del trabajo y se ajusta a criterios de legalidad y sostenibilidad académica. De forma complementaria, Google Earth Engine queda planteado únicamente como una herramienta potencial para análisis auxiliares sobre datasets con licencia explícita y resolución adecuada, pero no como fuente de adquisición de imágenes de alta resolución procedentes de mapas base de Google. Este enfoque proporciona un equilibrio adecuado entre precisión espacial, escalabilidad, trazabilidad técnica y rigor legal.

4.3. Comparativa de extensiones para VS Code y motor de la BBDD

Para el desarrollo de este proyecto la elección de la herramienta utilizada para la gestión y exploración de la base de datos resulta especialmente relevante. En este Trabajo Fin de Grado, dicha elección no se limita únicamente al sistema gestor de bases de datos, sino también a la extensión empleada dentro del entorno de desarrollo integrado. Una herramienta adecuada debe facilitar la inspección de esquemas, la ejecución de consultas y la depura-

ción de datos, sin introducir una complejidad innecesaria ni dependencias excesivas.

Entre las alternativas más consolidadas del estado del arte actual destaca, en primer lugar, SQLTools, una extensión de carácter modular que actúa como núcleo común al que se añaden controladores específicos según el motor de base de datos utilizado. Este enfoque permite trabajar de forma homogénea con diferentes sistemas, como SQLite, PostgreSQL o MySQL, sin modificar la herramienta principal. Desde un punto de vista arquitectónico, esta solución resulta especialmente atractiva en proyectos que pueden evolucionar con el tiempo, ya que evita la dependencia temprana de un único motor. No obstante, esta flexibilidad conlleva una ligera sobrecarga conceptual, puesto que el usuario debe gestionar explícitamente la instalación y configuración de los controladores necesarios.

Como alternativa especializada, la extensión SQLite, desarrollada por alexcvzz, está orientada exclusivamente a la gestión de este tipo de bases de datos, caracterizadas por funcionar como un único archivo local y no requerir de un servidor dedicado. Su propuesta prioriza la simplicidad y la inmediatez, ofreciendo un explorador visual de tablas, ejecución directa de sentencias SQL y opciones de exportación de resultados, todo ello con una configuración mínima. Esta aproximación resulta especialmente adecuada en fases iniciales del desarrollo, donde la base de datos se materializa como un fichero local y se requiere una herramienta ligera que no interfiera en el flujo de trabajo. La principal limitación de esta extensión radica en su carácter específico, ya que una eventual migración a otro motor implicaría adoptar una herramienta diferente.

Otra opción relevante es Database Client de Weijan Chen, una extensión que aspira a funcionar como un cliente de bases de datos completo dentro de Visual Studio Code, soportando múltiples motores e incluso conexiones avanzadas mediante túneles SSH. Aunque su nivel de funcionalidades es elevado, existen consideraciones importantes desde el punto de vista académico, como su transición hacia un modelo parcialmente cerrado y la presencia de mecanismos de telemetría. En el contexto de un Trabajo Fin de Grado, donde se valora la transparencia de las herramientas y la reproducibilidad del entorno, estos aspectos reducen su idoneidad como opción principal.

Finalmente, la extensión PostgreSQL for VS Code de ms-ossdata ofrece un conjunto de herramientas específicas para trabajar con PostgreSQL, tanto en entornos locales como en despliegues en la nube. Se trata de una solución robusta cuando el proyecto tiene claramente definido este motor como destino final. Sin embargo, su especialización la hace poco adecuada

en fases tempranas del desarrollo, especialmente cuando se trabaja con SQLite como base de datos embebida, ya que obliga a combinar múltiples extensiones para cubrir distintas necesidades.

Tras analizar las distintas alternativas, se ha optado por emplear SQLite como sistema gestor de bases de datos durante el desarrollo del proyecto, apoyándose en una extensión ligera y especializada para su gestión dentro de Visual Studio Code. Esta decisión responde a varios criterios fundamentales. En primer lugar, SQLite permite trabajar con una base de datos autocontenida, sin necesidad de desplegar servicios adicionales ni configurar infraestructuras externas, lo que simplifica el entorno de desarrollo y reduce la fricción inicial. En segundo lugar, su integración natural con aplicaciones web ligeras y con frameworks como Flask lo convierte en una solución especialmente adecuada para proyectos académicos de alcance controlado.

Desde la perspectiva del entorno de desarrollo, el uso de una extensión específica para SQLite ofrece una experiencia directa, coherente y alineada con los objetivos del proyecto, facilitando la inspección de datos y la validación del modelo relacional sin introducir complejidades innecesarias. Además, esta elección no compromete la evolución futura del sistema, ya que el diseño de la capa de persistencia permite, llegado el caso, una migración hacia motores más robustos como PostgreSQL, apoyándose entonces en herramientas más generalistas.

En consecuencia, la combinación de SQLite como motor de base de datos y una extensión dedicada en Visual Studio Code representa la opción con mejor equilibrio entre simplicidad y eficiencia, garantizando un desarrollo controlado y sostenible.

Aspecto	Bootstrap	Tailwind CSS
Enfoque	Conjunto de componentes predefinidos y estilos de alto nivel	Clases utilitarias de bajo nivel que permiten construir componentes a medida
Curva de aprendizaje	Accesible al inicio gracias a los componentes estándar y a los ejemplos completos	Inicialmente más exigente, requiere pensar en términos de propiedades y sistemas de diseño
Productividad inicial	Elevada en prototipos y desarrollos con requisitos visuales genéricos	Crece con el tiempo, especialmente cuando se definen patrones reutilizables y componentes propios
Personalización del diseño	Personalización profunda costosa, necesidad frecuente de sobrescribir estilos y romper el diseño base	Alta capacidad de personalización mediante combinaciones de utilidades y configuración centralizada
Mantenibilidad	Riesgo de hojas CSS voluminosas y difíciles de refactorizar en proyectos muy adaptados	Estilos más controlados, menor necesidad de sobrescrituras y reducción de la deuda técnica
Rendimiento y tamaño del CSS	Tamaño mayor en el fichero final cuando se utilizan muchos componentes sin optimización específica	Tamaño significativamente reducido gracias a la purga de utilidades no utilizadas en el producto final
Identidad visual	Interfaces con aspecto reconocible y similar a otros proyectos basados en el mismo <i>framework</i>	Mayor capacidad para definir una identidad visual propia y diferenciada
Comunidad y ecosistema	Comunidad muy amplia, gran cantidad de plantillas, ejemplos y recursos formativos	Comunidad en fuerte crecimiento, ecosistema consolidado en proyectos modernos basados en componentes

Tabla 4.1: Comparativa de ventajas e inconvenientes entre Bootstrap y Tailwind CSS.

Alternativa	Ventajas principales	Limitaciones
Google Earth Engine	Gran catálogo de datos científicos y alto rendimiento en análisis masivo	Resolución insuficiente en catálogos satelitales abiertos para trazas finas y ausencia de acceso exportable a imágenes Google Earth o Google Maps para análisis automático
Sentinel Hub / Copernicus	Integración sencilla mediante API y procesado bajo demanda	Resolución limitada a sensores Sentinel y dependencia de cuotas
Ortofoto aérea (PNOA)	Alta resolución sub-métrica, fuentes oficiales, licencia compatible con reutilización y buena reproducibilidad	Cobertura limitada al ámbito nacional y dependencia de los servicios oficiales disponibles
Google Maps / Earth imagery	Excelente calidad visual para visualización	Restricciones de licencia, extracción, almacenamiento y uso no adecuado para análisis automático
Folium / Leaflet	Facilidad en prototipos desde Python y alta flexibilidad en visores web interactivos	Folium resulta menos flexible en aplicaciones web avanzadas, mientras que Leaflet requiere mayor integración manual en el frontend

Tabla 4.2: Comparativa de motores y fuentes de datos geoespaciales.

Extensión	Ventajas principales	Limitaciones
SQLTools	Modular, soporta múltiples motores y facilita la evolución del proyecto	Requiere gestión de controladores y mayor configuración inicial
SQLite (alexcvzz)	Simplicidad, integración directa con ficheros SQLite y bajo coste cognitivo	Limitada a SQLite, no válida para otros motores
Database Client	Muy completa y con soporte para múltiples tecnologías	Modelo parcialmente cerrado y menor transparencia
PostgreSQL for VS Code	Herramientas específicas y maduras para PostgreSQL	Inadecuada para trabajar con SQLite

Tabla 4.3: Comparativa de extensiones para la gestión de bases de datos en Visual Studio Code

5. Aspectos relevantes del desarrollo del proyecto

Análisis del *notebook* de inferencia original

Un aspecto relevante del desarrollo fue la comprensión y adaptación del código procedente del trabajo de investigación en el que se basa este proyecto [15]. Antes de integrar la inferencia en la aplicación web, fue necesario estudiar el *notebook* y el flujo experimental proporcionado, identificando cómo se preparaban las imágenes, cómo se cargaban los modelos, cómo se ejecutaban las predicciones y cómo se transformaban las salidas del modelo en máscaras de segmentación interpretables.

Este trabajo previo permitió trasladar un código orientado a experimentación a un entorno web con interacción real de usuario, control de errores, persistencia de resultados y ejecución repetible. Durante este proceso se analizaron distintas posibilidades de integración, separando las partes reutilizables del *pipeline* original de aquellas que debían adaptarse para funcionar dentro de Flask, con rutas, formularios, ficheros subidos, modelos activos y procesamiento en segundo plano. Esta fase resultó fundamental para que la aplicación no se limitase a ejecutar un modelo, sino que incorporase la lógica de inferencia de forma coherente, mantenible y alineada con el comportamiento validado en el artículo original.

Comparativa de despliegue: modelos *pickle* vs PyTorch

Durante el desarrollo se plantearon dos formas de ejecutar los modelos entrenados: por un lado, cargar el objeto completo serializado con `pickle` y, por otro, exportar el modelo a una representación TorchScript, ejecutable

con el runtime de PyTorch, orientada a inferencia. La motivación inicial de la exportación a PyTorch fue principalmente de despliegue: un módulo TorchScript permite ejecutar la red sin depender del código de entrenamiento, reduce acoplamientos con *frameworks*, como, por ejemplo, mediante PyTorch Lightning y facilita integrar el modelo en otros entornos de ejecución.

Sin embargo, en esta aplicación el criterio prioritario es la calidad de segmentación de las trazas. En la práctica se observó que, aun empleando la misma arquitectura y pesos por modelo, el resultado de segmentación entre el que es cargado con `pickle` y el exportado a TorchScript no es exactamente idéntico. Estas diferencias pueden estar motivadas por detalles de compatibilidad entre versiones debido a cambios internos en `segmentation_models_pytorch`, por pequeñas variaciones numéricas del grafo trazado y por el encapsulado de normalización durante la exportación.

Metodología de evaluación

Para comparar ambos enfoques se midieron dos aspectos:

- **Rendimiento:** tiempo de carga del modelo por *fold* (`load_ms`) y tiempo de inferencia medio (`infer_ms_mean`) sobre la misma imagen.
- **Equivalencia de salida:** similitud entre máscaras binarias mediante IoU (`mask_iou`) y Dice (`mask_dice`).

Resultados principales

La Tabla 5.1 recoge los resultados por *fold*. En términos de equivalencia, los valores de *Dice* se sitúan aproximadamente entre 0.96 y 0.98, e IoU alrededor de 0.92 y 0.96. Conviene destacar que, en este caso, *IoU* y *Dice* no evalúan la segmentación frente a una anotación, sino que cuantifican el solapamiento entre las dos máscaras predichas: la generada por el modelo cargado con `pickle` y la generada por el modelo en TorchScript. Estos resultados indican que ambos *pipelines* producen máscaras muy parecidas, pero no idénticas, existiendo discrepancias localizadas, como bordes finos o algunas trazas débiles, que en este problema pueden ser relevantes.

En rendimiento, TorchScript presenta en general tiempos de carga inferiores a `pickle` en la mayoría de *folds*. Sin embargo, la carga se ejecuta una vez al inicializar el sistema, mientras que la inferencia se repite para cada imagen. En esta, los tiempos medios (`infer_ms_mean`) resultan comparables entre ambos enfoques y no muestran una ventaja suficiente como para justificar

sacrificar calidad de segmentación. Además, la dispersión (`infer_ms_std`) es similar, lo que sugiere estabilidad de tiempos en ambos casos.

Fold	Pickle load (ms)	Torch load (ms)	Pickle infer (ms)	Torch infer (ms)	IoU	Dice
0	9 688	2 582	11 670	12 658	0,945 6	0,972 0
1	11 911	2 176	12 172	12 020	0,940 2	0,969 2
2	16 182	2 024	11 427	11 189	0,924 4	0,960 7
3	3 221	1 850	11 992	9 148	0,940 1	0,969 1
4	5 671	2 121	10 151	9 505	0,952 7	0,975 8
5	30 966	1 421	9 687	9 396	0,958 1	0,978 6
6	3 187	2 110	9 057	9 505	0,950 7	0,974 7
7	3 185	1 702	9 748	9 443	0,956 9	0,977 9
8	3 655	2 171	9 378	9 542	0,953 3	0,976 1
9	3 552	1 612	9 099	9 268	0,949 6	0,974 1

Tabla 5.1: Comparativa por *fold* entre carga e inferencia usando modelos serializados con `pickle` y exportados a TorchScript.

Decisión de diseño

Aunque TorchScript constituye una alternativa robusta para despliegue, al reducir la dependencia respecto al código Python de entrenamiento y facilitar la portabilidad del modelo, en el contexto de este trabajo se prioriza mantener la coherencia con el comportamiento validado durante el entrenamiento y la evaluación original.

Para ello se toma como referencia la ruta de carga del modelo mediante `pickle`, no porque `pickle` realice la inferencia, sino porque permite reconstruir el modelo heredado tal y como fue utilizado en el *pipeline* original. La inferencia, en todos los casos, sigue realizándose mediante PyTorch sobre el modelo cargado.

Esta decisión se justifica porque:

- las salidas obtenidas mediante la ruta basada en `pickle` y la versión exportada a TorchScript no son exactamente iguales ($\text{IoU}/\text{Dice} < 1$);
- el coste temporal de inferencia es similar entre ambas alternativas;
- las diferencias en el tiempo de carga no compensan una posible variación en la clasificación píxel a píxel, especialmente en estructuras finas como las trazas.

Por ello, el sistema mantiene como referencia los modelos heredados cargados mediante `pickle`, garantizando coherencia con los resultados esperados del *pipeline* original. No obstante, la aplicación conserva compatibilidad con modelos exportados a TorchScript u otros artefactos PyTorch compatibles, siempre que superen la validación técnica previa a su incorporación.

Problema de licencias

La elección de la fuente cartográfica y del flujo de adquisición de imágenes estuvo condicionada por restricciones de licenciamiento y de uso impuestas por los proveedores. Aunque Google Earth Engine (GEE) proporciona un entorno potente para el análisis geoespacial, su utilización no implica un derecho automático a extraer y reutilizar imágenes de cualquier capa visualizable ni a tratar los mapas base como un repositorio libre de datos [22]. En particular, los términos y políticas asociados a servicios de cartografía de Google limitan expresamente la extracción, descarga masiva, almacenamiento y, de forma relevante para este proyecto, el uso del contenido con fines de análisis automático, como la detección o segmentación de estructuras [25].

Dado que el objetivo del sistema requiere obtener recortes de alta resolución y someterlos a un *pipeline* de segmentación semántica, basar la adquisición de imágenes en capturas de mapas base con restricciones habría comprometido la reproducibilidad y la conformidad legal del desarrollo.

Como consecuencia, se optó por un enfoque alternativo que garantizase seguridad jurídica y trazabilidad técnica: el uso de ortofotografía pública con licencia abierta y adecuadamente atribuible. En este contexto se seleccionó el Plan Nacional de Ortofotografía Aérea (PNOA) [32, 9] como fuente principal de imágenes, integrándolo con un visor web construido con Leaflet [1] y un mapa base de OpenStreetMap (OSM) [47] únicamente para la interacción del usuario y la selección del área de interés.

Procesos asíncronos en segundo plano

Otra decisión relevante del desarrollo ha sido separar determinadas operaciones pesadas del ciclo inmediato de petición y respuesta HTTP. En una aplicación web, mantener al usuario esperando mientras el servidor completa tareas de larga duración perjudica la experiencia de uso, incrementa el riesgo de *timeout* y bloquea recursos que deberían quedar disponibles para atender nuevas solicitudes.

Por este motivo, el sistema incorpora un *worker* de trazas que permite desacoplar la creación de una colección geoespacial del procesamiento com-

pleto de sus teselas. Cuando el usuario define una zona en el visor, el sistema registra la parcela y sus fotos asociadas en estado **pending**; posteriormente, el *worker* reclama cada imagen, la marca como **processing**, materializa la tesela si es necesario, ejecuta la inferencia con PyTorch, guarda el fichero JSON de trazas y actualiza el estado final a **completed** o **failed**. De este modo, la aplicación puede responder rápidamente a la acción inicial y mostrar la colección creada, mientras el avance se consulta de forma progresiva mediante los estados persistidos. Asimismo, se incorporó un mecanismo de pausa del procesamiento, accesible desde la interfaz, para permitir detener temporalmente la segmentación de teselas sin finalizar de forma abrupta los procesos de la aplicación ni comprometer la consistencia de los estados almacenados.

Para que este procesamiento pudiera escalar a varios *workers* concurrentes, se modificó también la forma de reclamar teselas pendientes. El enfoque inicial, basado en buscar fotos en estado **pending** y marcarlas después como **processing**, resultaba suficiente con un único proceso, pero podía generar condiciones de carrera si dos *workers* leían la misma tesela antes de que su estado quedase actualizado. Para evitarlo, la reclamación se convirtió en una operación atómica sobre SQLite: se abre una transacción corta mediante **BEGIN IMMEDIATE**, se seleccionan candidatas en estado **pending** o **processing** antiguo, se actualizan de forma condicionada por estado, se incrementa **numero_intentos** y se confirma la transacción inmediatamente. La inferencia queda fuera de esta transacción, de forma que el bloqueo sobre la base de datos se mantiene el menor tiempo posible. Así, SQLite actúa como una cola persistente ligera, capaz de conservar el estado de las teselas y recuperar trabajos *stale* que hubieran quedado interrumpidos.

Este cambio mantiene la lógica de pausa y reanudación ya existente. La pausa continúa representándose mediante un estado de pausa, sin añadir una nueva columna específica, y las teselas en estado **processing** de esa parcela pueden devolverse a **pending** para ser retomadas más adelante. Los *workers* no reclaman teselas pertenecientes a parcelas pausadas y, si una tesela finaliza mientras la parcela ha sido pausada, el resultado no se marca indebidamente como completado. Del mismo modo, una colección ya completada no puede pausarse, al no existir trabajo pendiente que detener. Además, el número de *workers* concurrentes queda parametrizado desde el fichero **.env**, permitiendo ajustar el grado de paralelismo del despliegue sin modificar el código de la aplicación.

El segundo proceso asíncrono corresponde a la validación de modelos desde el panel de administración. Cuando se incorpora un nuevo modelo, el

sistema no lo habilita de forma inmediata, sino que lo registra inicialmente como pendiente y lanza una tarea de validación en segundo plano. Esta tarea comprueba que el archivo puede cargarse correctamente, que es compatible con el flujo de inferencia y que supera una prueba funcional antes de marcarlo como validado. Si la validación falla, el sistema elimina el artefacto, descarta el registro asociado y deja constancia del error para informar al administrador. Esta estrategia evita que la subida de modelos bloquee el proceso web, mejora la robustez ante ficheros incompatibles y permite mantener un control explícito sobre qué modelos pueden utilizarse para generar trazas.

En conjunto, ambos procesos asíncronos refuerzan el rendimiento, la trazabilidad y la tolerancia a fallos del sistema, al permitir que las operaciones costosas continúen en segundo plano sin comprometer la respuesta inmediata de la interfaz.

Despliegue containerizado con Docker y Gunicorn

El despliegue final de la aplicación se ha realizado mediante Docker Compose, separando el servicio web del servicio encargado del procesamiento de trazas. El contenedor **web** ejecuta la aplicación Flask mediante Gunicorn y atiende las peticiones HTTP del navegador, mientras que el contenedor **worker** se encarga de procesar las colecciones en segundo plano. Ambos servicios comparten los mismos volúmenes persistentes, donde se almacenan la base de datos SQLite, los modelos, las imágenes, las teselas y los resultados generados, evitando que estos recursos queden incluidos dentro de la imagen Docker.

Esta separación permite reproducir el entorno de despliegue de forma más controlada y mantener una división clara entre la parte interactiva de la aplicación y las tareas pesadas de inferencia. En producción, el servicio web se configura con `AUTO_START_TRACE_WORKER=false`, de modo que no arranca procesos de trazas internos y delega dicha responsabilidad en el servicio **worker**. Además, el número de contenedores de procesamiento y su concurrencia interna pueden ajustarse mediante Docker Compose y variables de entorno, permitiendo adaptar el despliegue a los recursos disponibles del servidor sin modificar el código de la aplicación.

Compatibilidad de PyTorch y CUDA en el despliegue

En el entorno general del proyecto, el fichero `requirements.txt` mantenía versiones recientes de las dependencias principales, entre ellas `torch==2.10.0`

y `torchvision==0.25.0`. Debido a ello, en la imagen Docker inicial de despliegue se instaló `torch 2.10.0+cu128`, asociado al *runtime* CUDA 12.8. Sin embargo, al probarlo sobre el servidor Gamma, equipado con tres GPU NVIDIA TITAN Xp, PyTorch advertía de que dichas tarjetas tienen capacidad CUDA `sm_61`, mientras que esa compilación solo soportaba arquitecturas a partir de `sm_70`. Por tanto, aunque `torch.cuda.is_available()` pudiera devolver `True`, no era una configuración segura para ejecutar inferencia real sobre GPU.

Para resolverlo, se ajustó el entorno Docker de despliegue instalando versiones compatibles con el hardware disponible: `torch 2.7.1+cu118` y `torchvision 0.22.1+cu118`. Tras este cambio se comprobó que el contenedor detectaba correctamente las tres GPU, que podía ejecutar operaciones reales en CUDA y que la inferencia quedaba preparada para utilizar aceleración por GPU. Este ajuste afecta únicamente al *runtime* PyTorch/CUDA del despliegue, sin modificar el modelo de segmentación ni sus pesos.

6. Trabajos relacionados

Este capítulo presenta una revisión de los principales trabajos y herramientas relacionados con el problema abordado en este TFG. Por un lado, se analizan artículos científicos recientes que aplican técnicas de teledetección y aprendizaje profundo a la detección de estructuras lineales en entornos naturales, con especial atención a sendas de fauna y caminos poco definidos en imágenes aéreas o satelitales. Por otro lado, se describen herramientas software ampliamente utilizadas para la visualización y análisis de imágenes remotas, relevantes en contextos de ecología del paisaje y conservación.

Esta revisión permite situar el proyecto dentro del estado del arte, identificar enfoques metodológicos afines y justificar las decisiones técnicas adoptadas a lo largo del desarrollo.

6.1. Artículos científicos relacionados

En este apartado se revisan los avances recientes en segmentación semántica aplicada a la detección de estructuras lineales, como lo son las sendas de fauna o los caminos rurales en imágenes aéreas o satelitales, especialmente en contextos ecológicos y de conservación. Se incluyen trabajos enfocados en el uso de aprendizaje profundo mediante redes neuronales convolucionales para identificar tales patrones lineales en terreno natural, con casos de estudio internacionales.

- **Remotely-Sensed Game Trails Are a Behavioral Footprint That Explains Patterns of Herbivore Habitat Use [61]**

Este estudio pionero analiza sendas de fauna en la sabana africana usando imágenes aéreas para inferir el uso de hábitat por herbívoros.

Los autores cuantificaron la densidad de sendas mediante la fórmula: m/ha a partir de ortoimágenes de alta resolución y la modelaron frente a variables ambientales. Encontraron que la cobertura de vegetación leñosa es el mejor predictor: las mayores densidades de sendas ocurren con coberturas intermedias de matorral. Las sendas detectadas mostraron altas correlaciones con otros indicadores de uso de hábitat, como por ejemplo, los conteos de excrementos, validando su utilidad como huella del comportamiento de herbívoros a escala de paisaje. Este trabajo revela por primera vez que las sendas visibles en imágenes son un indicador integrador de los patrones de movimiento de fauna, proporcionando datos novedosos para ecología del paisaje y conservación.

- **Human Inspired Deep Learning to Locate and Classify Terrestrial and Arboreal Animals in Thermal Drone Surveys [2]**

Enfocado en monitoreo de fauna por drones, este artículo emplea técnicas de deep learning inspiradas en la percepción humana para detectar y clasificar animales terrestres y arborícolas en imágenes térmicas aéreas. Se desarrolló una red neuronal convolucional entrenada con vuelos de dron en Nueva Gales del Sur, Australia, para identificar especies crípticas como koalas y petauros en imágenes nocturnas. El modelo logró resultados robustos en la detección multiespecífica, demostrando el potencial de las *CNN* para procesamiento autónomo de imágenes térmicas en programas de monitoreo de vida silvestre a gran escala.

- **Automized Segmentation of Animal Paths in UAS Imagery Using Deep Learning — A Case Study from the Kruger National Park [42]**

Este trabajo combina drones y aprendizaje profundo para mapear las rutas de paso de fauna en la sabana africana. Se obtuvieron mosaicos ortofotográficos RGB de alta resolución a través del uso de drones en el Parque Kruger, en Sudáfrica y se entrenó una *UNet* para segmentar automáticamente la red de sendas generadas por animales. Obtuvieron hasta un 83 % de precisión en la detección de sendas, demostrando la eficacia del modelo aún cuando muchas sendas son prácticamente invisibles al ojo humano. Un desafío destacado fue la generación de datos de entrenamiento de calidad: los autores implementaron un método semiautomático para etiquetar sendas, estimando su anchura a partir de datos de elevación fotogramétrica, lo que mejoró sustancialmente el rendimiento del modelo. Este estudio, fruto de la colaboración en-

tre la Universidad de Würzburg y Parques Nacionales sudafricanos, muestra una herramienta escalable para investigar corredores de movimiento animal sin recurrir a collares GPS, complementando métodos tradicionales con mapas continuos de sendas.

- **Mapping Trails and Tracks in the Boreal Forest Using LiDAR and Convolutional Neural Networks** [40]

En ecosistemas boreales de Canadá, este estudio mapea sendas y huellas en bosques boreales, tanto de fauna silvestre como senderos de vehículos creados por todoterrenos, empleando una combinación innovadora de *LiDAR* de alta densidad y *CNN*. Se utilizó un modelo *U-Net* entrenado sobre modelos digitales de terreno (*DTM*) de 10 cm de resolución obtenidos con *LiDAR* montado en dron, complementado con datos *LiDAR* aéreos de 50 cm. El modelo logró identificar automáticamente una extensa red de 2.829 km de senderos en un área de 59 km² de bosque boreal, con alta exactitud, obteniendo: $F_1 \approx 0,74-0,77$. Se observó que las sendas se concentran especialmente en turberas y que a menudo aprovechan perturbaciones lineales humanas como parte de la red de movilidad. Este trabajo subraya la escasez previa de literatura sobre detección remota de sendas, demostrando el potencial de fusionar sensores activos *LiDAR* con *Deep Learning* para cartografiar rasgos lineales sutiles en ambientes naturales a gran escala, lo que puede informar tanto la gestión de fauna como la restauración de disturbios.

- **Estimating Forest Biomass Using Small Footprint LiDAR Data: An Individual Tree-Based Approach That Incorporates Training Data** [5]

Aunque enfocado en biomasa forestal y no en sendas, se incluye este trabajo clásico por su relevancia metodológica en el uso de datos *LiDAR* de alta resolución. Los autores desarrollaron un algoritmo a nivel de árbol biológico individual para estimar biomasa aérea mediante *small-footprint LiDAR*, demostrando la factibilidad de predecir variables ecológicas con alta precisión espacial. Aplicaron regresiones entre la densidad de retornos *LiDAR* por árbol y mediciones de biomasa, logrando resultados prometedores. Si bien este estudio no trata sendas, sentó bases en el procesamiento de datos *LiDAR* para estructuras ecológicas, técnica que, como muestran en McDermid et al. [40], hoy facilita la detección de rasgos lineales sutiles, como sendas o cauces, en entornos naturales. Es un ejemplo temprano de cómo combinar teledetección detallada con algoritmos especializados puede extraer información ecológica no evidente mediante métodos tradicionales.

- **Deep learning- and image processing-based methods for automatic estimation of leaf herbivore damage [70]**

En un dominio distinto pero conceptualmente afín, los autores proponen un método automático para estimar el daño por herbivoría en hojas individuales a partir de imágenes, con el objetivo de sustituir procedimientos manuales lentos y potencialmente sesgados. Su enfoque combina procesamiento de imagen con un modelo generativo adversario condicionado, basado en una variante de *pix2pix* [33], que reconstruye la hoja como si estuviese intacta a partir de la versión dañada, de modo que el área consumida se infiere comparando la imagen original con la reconstrucción generada.

Desde una perspectiva metodológica, este trabajo resulta relevante para el presente TFG porque evidencia que, en problemas ecológicos, el aprendizaje profundo permite cuantificar automáticamente huellas de herbivoría con resultados precisos y salidas visualmente interpretables. No obstante, la escala y la finalidad difieren, ya que aquí se persigue delimitar espacialmente estructuras lineales difusas en ortoimágenes mediante segmentación semántica, en lugar de estimar un porcentaje de daño foliar. Aun así, ambas líneas convergen en una misma motivación, trasladar técnicas avanzadas de visión por computador a herramientas utilizables por personal no especializado, algo que Wang et al. materializan mediante una aplicación web, y que este proyecto persigue al encapsular la inferencia y la visualización de sendas en una interfaz interactiva orientada a usuario final.

Trabajo	Objetivo	Datos	Enfoque / Modelo	Salida	Aportación y relación con el TFG
Stears et al. (2025) [61]	Inferir uso de hábitat a partir de redes de sendas y su densidad.	Ortoimágenes de alta resolución en sabana africana.	Cuantificación de densidad de sendas (m/ha) y modelado frente a covariables ambientales.	Métricas de densidad y relaciones ecológicas.	Justifica el valor ecológico de las sendas como huella observable y motivación para su detección automática.
Backman et al. (2025) [2]	Detección y clasificación de fauna en campañas con dron.	Imágenes térmicas UAS en Australia.	<i>Deep learning</i> con <i>CNN</i> para detección multiespecie.	Localización y clase de individuos.	Ejemplifica automatización en monitorización ecológica y despliegue práctico de visión por computador.
Müller et al. (2025) [42]	Segmentación automática de sendas de fauna en terreno natural.	Mosaicos RGB UAS, Parque Kruger (Sudáfrica).	Segmentación semántica con <i>U-Net</i> ; etiquetado semiautomático apoyado en elevación fotogramétrica.	Máscara de sendas y red continua.	Antecedente directo por formulación como segmentación densa de sendas, alineado con el <i>pipeline</i> del TFG.
McDermid et al. (2025) [40]	Mapeo de senderos y huellas en entornos boreales.	<i>LiDAR</i> alta densidad y DTM, Canadá.	<i>U-Net</i> sobre DTM multi-resolución; extracción de rasgos lineales sutiles.	Red de senderos; $F_1 \approx 0,74-0,77$.	Evidencia que la combinación de sensores y <i>deep learning</i> cartografía estructuras lineales débiles a gran escala.
Bortolot & Wynne (2005) [5]	Estimación de biomasa forestal a nivel de árbol.	<i>Small-footprint LiDAR</i> .	Modelado individual por árbol con datos de entrenamiento.	Biomasa estimada por árbol.	Base metodológica en explotación de <i>LiDAR</i> ecológico, relevante como antecedente instrumental, no centrado en sendas.
Wang et al. (2024) [70]	Estimación automática de daño foliar por herbivoría.	Imágenes de hojas individuales.	GAN condicionada tipo <i>pix2pix</i> para reconstrucción y cálculo de área perdida.	Porcentaje de daño y salida visual reconstruida.	Relacionado por la cuantificación automática de huellas de herbivoría y su orientación a usuario final mediante aplicación web.

Tabla 6.1: Comparativa de trabajos relacionados según objetivo, datos, enfoque metodológico y relación con el presente TFG.

6.2. Herramientas Visuales para Imágenes Satelitales y Aéreas

En paralelo a los avances científicos, existen diversas herramientas de software que facilitan el acceso, visualización y análisis de imágenes aéreas y satelitales. A continuación se describen las más relevantes para proyectos de ecología del paisaje, excluyendo Leaflet y Google Earth Engine, ya que ya han sido abordadas en este TFG. Cada herramienta se caracteriza por su propósito, funciones clave, tipos de datos soportados y posibles usos en la detección de estructuras lineales naturales:

■ Interfaz de QGIS [51]

Un Sistema de Información Geográfica open source¹ de escritorio, ampliamente utilizado por su potencia y flexibilidad. QGIS permite visualizar y procesar prácticamente cualquier formato ráster o vectorial, incluyendo extensiones como: GeoTIFF, JPEG2000, Shapefiles, etc..., donde se encuentran ortofotos aéreas y mosaicos satelitales. Ofrece herramientas de GIS avanzadas, siendo algunas de estas: la georreferenciación, el análisis de terreno y los filtros espaciales, y puede integrarse con datos remotos mediante complementos. Por ejemplo, el plugin *QuickMapServices* [46]² brinda acceso directo a mapas base satelitales de diversas fuentes: Google, Bing, OSM, etc..., y el plugin *Sentinel Hub* [56]³ conecta con la plataforma Copernicus.

QGIS es ideal para proyectos de conservación pues permite superponer capas y realizar análisis espaciales en un entorno unificado, como se muestra en la imagen: 6.1. Su capacidad de *scripting* mediante Python y sus múltiples plugins facilitan la automatización de tareas como detección de caminos rurales o generación de mapas de densidad de sendas. Al ser software libre, es altamente personalizable y cuenta con una gran comunidad, con casos de uso en administraciones ambientales, universidades de España para gestión de hábitats y planificación territorial.

■ USGS EarthExplorer [67]

¹<https://qgis.org/>

²https://plugins.qgis.org/plugins/quick_map_services/

³<https://www.sentinel-hub.com/develop/integrate/desktopgis/qgis-plugin/>

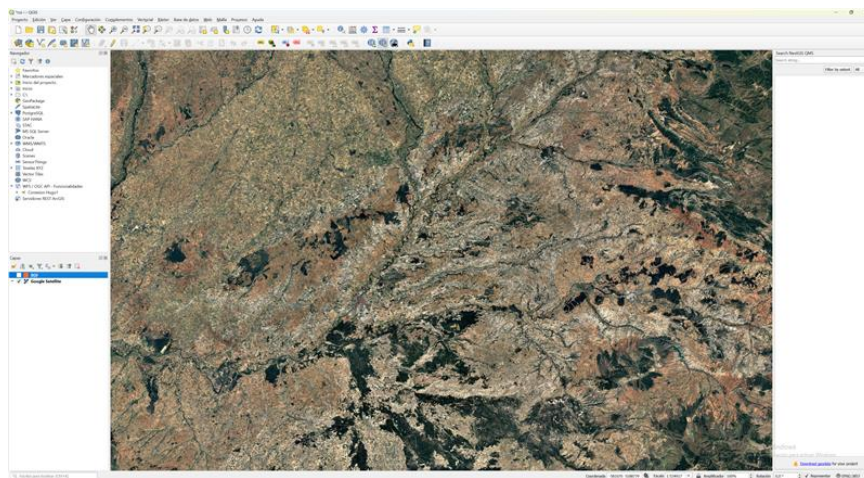


Figura 6.1: Interfaz de la herramienta QGIS. Se visualiza un mapa de Google Satellite ya cargado.

Es el portal web de la agencia geológica estadounidense (USGS)⁴ para buscar, visualizar y descargar imágenes de satélite y fotografía aérea de múltiples fuentes. EarthExplorer funciona como una interfaz de descubrimiento de datos: el usuario delimita un área de interés en un mapa interactivo, define rangos de fechas y tipos de producto, como, por ejemplo, colecciones Landsat, Sentinel-2 y fotografías aéreas históricas, y el sistema devuelve las escenas disponibles, tal y como se aprecia en la imagen: 6.2. Incluye opciones avanzadas de filtrado, siendo algunas: por cobertura de nubes, ángulo solar, etc..., y permite examinar imágenes de baja resolución antes de ordenar la descarga. Es necesario un registro gratuito para obtener los datos.

EarthExplorer da acceso a millones de imágenes de archivo desde los años 1960 en adelante, incluyendo datos de libre acceso como Landsat de niveles L1/L2, imágenes desclasificadas de satélites espía (CORONA), fotografías aéreas USDA, modelos digitales de elevación, entre otros. Su utilidad en proyectos ecológicos radica en compilar series temporales de imágenes históricas de un área. Un ejemplo aplicado a nuestro caso, podría ser el observar la apertura de nuevos caminos o sendas a lo largo de décadas y visualizar los cambios en cobertura vegetal asociados a herbívoros. También facilita obtener imágenes multiespectrales calibradas con las que aplicar algoritmos de segmentación entrenados en diferentes fuentes. Si bien EarthExplorer no permite un análisis geoespacial complejo por sí mismo, es un nodo central

⁴<https://earthexplorer.usgs.gov/>

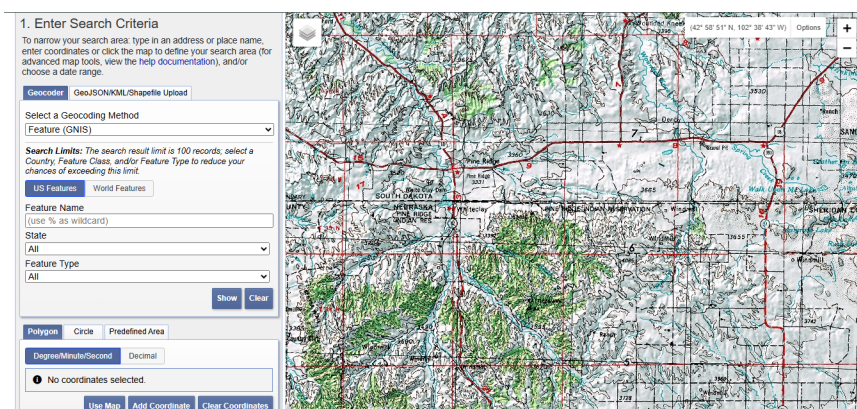


Figura 6.2: Vista de la interfaz web de USGS EarthExplorer.

de acceso a datos de teledetección clásicos indispensable en flujos de trabajo reproducibles.

■ Sentinel Hub EO Browser [55]

Plataforma web desarrollada por Sinergise⁵ que ofrece una visualización instantánea de datos satelitales en la nube, con énfasis en las misiones Sentinel, del programa Copernicus, y otros datasets abiertos. EO Browser permite explorar y comparar imágenes en alta resolución de múltiples fuentes, como: Sentinel-1, -2, -3, Landsat 5-9, MODIS, entre otros, de forma interactiva. El usuario puede elegir un área simplemente navegando por el mapa, seleccionar un rango de fechas y filtrar por un porcentaje de nubes. La aplicación carga en segundos las imágenes disponibles cumpliendo esos criterios y las muestra en pantalla. Ofrece también visualizaciones personalizables, como pudieran ser las combinaciones de bandas espectrales, los índices de vegetación NDVI/NDWI predefinidos, las capas de falso color, etc..., y herramientas integradas para descargar imágenes en alta resolución, generar animaciones timelapse multitemporales, o incluso ejecutar scripts de procesamiento personalizados sobre los píxeles.

EO Browser es una demo gratuita de las capacidades del servicio Sentinel Hub, por lo que no requiere al usuario instalar software ni manejar grandes volúmenes de datos localmente: todo el procesamiento se hace en servidores en tiempo real. Para proyectos de detección de sendas o caminos, EO Browser resulta muy útil en la fase exploratoria, ya que permite revisar rápidamente imágenes Sentinel-2 recientes de nuestro sitio de estudio, comparar épocas, identificar visualmente

⁵<https://apps.sentinel-hub.com/eo-browser/>

posibles patrones lineales y acotar períodos de interés. Si se necesita análisis más profundo, las imágenes encontradas se pueden descargar para tratamiento en QGIS u otro entorno. En resumen, EO Browser acerca el *big data* satelital a cualquier usuario: con unos clics, se accede a décadas de imágenes globales, lo que empodera proyectos de conservación con información actualizada casi en tiempo real, ya que Sentinel-2 tiene una revisita de 5 días y los datos suelen estar disponibles horas después de su adquisición.

- **Copernicus Browser** [12]

Visor en línea oficial del programa Copernicus de la Agencia Espacial Europea, útil para acceder a los datos Sentinel de forma sencilla. Lanzado en 2023 dentro del Copernicus Data Space Ecosystem⁶, este navegador gratuito y abierto presenta una experiencia muy similar a EO Browser, con enfoque en la facilidad de uso para el público general. Al ingresar, se muestra inmediatamente un mosaico sin nubes de Europa; el usuario puede navegar, hacer zoom y visualizar imágenes recientes prácticamente de cualquier lugar del mundo, como se muestra en la imagen 6.3. Con un par de clics mediante el botón: “Visualize”, el sistema obtiene la última imagen Sentinel-2 sin nubes de la zona visualizada. Copernicus Browser integra también datos de Sentinel-1 (radar), Sentinel-3 (medio ambiente) y Sentinel-5P (atmósfera), permitiendo alternar entre diferentes misiones. Incorpora herramientas interactivas como vista 3D del terreno, medición de distancias/áreas, comparación temporal de imágenes, con: *timelapse* y el deslizador antes-después y colecciones temáticas predefinidas, destacando fenómenos de interés, siendo algunos: la geología, la hidrología, los cambios urbanos, etc...

Un punto fuerte es que no requiere experiencia GIS: está diseñado para que incluso estudiantes o gestores no especializados exploren imágenes satelitales fácilmente. Esto resulta valioso en proyectos de biodiversidad donde se quiera involucrar a comunidades locales o personal de campo: por ejemplo, los guardabosques pueden abrir el navegador y ver rápidamente dónde han surgido nuevas sendas de ganado en una reserva, comparando con meses previos sin tener que procesar datos. Aunque no ofrece tantas opciones avanzadas como Sentinel Hub, ya que no se pueden ejecutar scripts personalizados, Copernicus Browser garantiza acceso abierto y streaming fluido de datos Copernicus, siendo excelente para visualización y educación

⁶<https://browser.dataspace.copernicus.eu/>

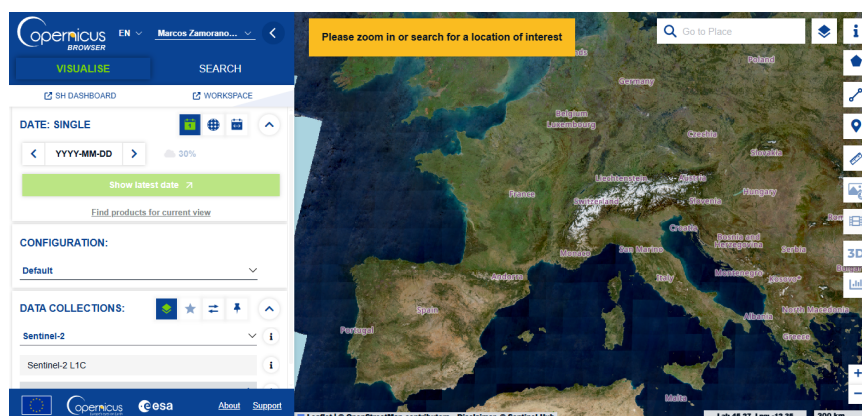


Figura 6.3: Vista de la interfaz web de Copernicus Browser. Se muestra un mosaico satelital en color real sobre Europa.

ambiental. Para análisis detallados, los usuarios pueden registrarse y descargar los productos originales Sentinel desde la misma interfaz. En resumen, es una herramienta intuitiva y poderosa para poner imágenes satelitales al alcance de cualquier interesado en monitorear el territorio.

■ Otras herramientas

Además de las anteriores, cabe mencionar brevemente otras plataformas útiles según la finalidad: ESA SNAP [19]⁷, que consiste en un software de escritorio para procesamiento avanzado de imágenes de radar y ópticas, muy usado en investigación, Google Earth Pro [24]⁸, que permite la visualización 3D de imágenes históricas, aunque con limitaciones de resolución temporal en áreas naturales, NASA Worldview [44]⁹, un visor web de imágenes globales diarias, útil para ver rápidamente incendios, inundaciones u otros eventos, o visores regionales como el Centro de Descargas del CNIG [8]¹⁰ en España.

No obstante, para este proyecto centrado en sendas ecológicas, las herramientas ya descritas, QGIS, EarthExplorer, Sentinel Hub EO Browser y Copernicus Browser, cubren un espectro completo: desde el análisis GIS exhaustivo offline, pasando por la búsqueda masiva de datos brutos, hasta la visualización interactiva online. Integrar estas herramientas en el flujo de trabajo permitirá identificar, segmentar y validar las estructuras lineales de interés con mayor eficacia y rigor.

⁷<https://step.esa.int/main/download/snap-download/>

⁸<https://www.google.com/earth/about/versions/>

⁹<https://worldview.earthdata.nasa.gov/>

¹⁰<https://centrodedescargas.cnig.es/>

7. Conclusiones y Líneas de trabajo futuras

En este capítulo se recogen las conclusiones obtenidas tras la finalización del proyecto, valorando el grado de cumplimiento de los objetivos planteados, las principales decisiones tomadas durante el desarrollo y el aprendizaje técnico adquirido a lo largo del proceso. Asimismo, se incluyen posibles líneas de trabajo futuras, orientadas a ampliar las funcionalidades del sistema y mejorar su aplicabilidad en escenarios reales de análisis geoespacial.

7.1. Conclusiones

A continuación se sintetizan las principales conclusiones extraídas tras la finalización del proyecto:

- Se han cumplido los objetivos establecidos inicialmente, trasladando la segmentación semántica de trazas de herbivoría a una aplicación web funcional. La solución permite cargar imágenes, ejecutar la detección, visualizar los resultados y gestionar flujos asociados a usuarios, colecciones geoespaciales y modelos.
- Se ha desarrollado una aplicación web monolítica modular con Flask, separando responsabilidades entre interfaz, lógica de aplicación, persistencia, procesamiento de imágenes e inferencia. Además del análisis de imágenes individuales, se ha incorporado soporte para visor cartográfico, generación de teselas, colecciones persistentes y procesamiento en segundo plano.

- Se ha diseñado un sistema extensible para la gestión de modelos, evitando que la aplicación dependa de uno único. Los administradores pueden incorporar modelos, validarlos antes de su uso y seleccionar cuál debe actuar como modelo activo, facilitando así la evolución futura del sistema.
- Ha sido necesario estudiar en profundidad tanto el código de ejecución del artículo base [15], como el algoritmo proporcionado, analizando el flujo de preparación de imágenes, carga de modelos, ejecución de predicciones y generación de máscaras antes de adaptarlo a una aplicación con interacción real de usuario.
- Se han aplicado conocimientos adquiridos en distintas asignaturas del Grado en Ingeniería Informática, especialmente en *Bases de Datos*, *Análisis y Diseño de Sistemas*, *Interacción Hombre-Máquina*, *Gestión de Proyectos*, *Estructuras de Datos*, *Algoritmia*, *Sistemas Inteligentes*, *Minería de Datos* y *Sistemas Distribuidos*.
- El proyecto también ha requerido adquirir nuevos conocimientos, principalmente en desarrollo web, peticiones y respuestas HTTP, gestión de sesiones, autenticación, autorización, formularios, renderizado con plantillas, comunicación mediante JSON, JavaScript, HTML, CSS y herramientas modernas de desarrollo.
- Uno de los retos principales ha sido transformar un entorno experimental, basado en un *notebook* de inferencia, en una aplicación funcional capaz de gestionar errores, estados intermedios, usuarios, ficheros, modelos, permisos, procesos asíncronos y persistencia de resultados.
- Finalmente, se ha completado la documentación asociada al proyecto, incluyendo la memoria y los anexos. Esta tarea ha permitido justificar decisiones, ordenar conceptos, explicar alternativas y dejar constancia suficiente para facilitar la evaluación, mantenimiento y posible continuación del sistema.

7.2. Líneas de trabajo futuras

A partir del trabajo realizado, las líneas de mejora identificadas son las siguientes:

- Una línea de mejora prioritaria sería estudiar la obtención de una licencia o autorización específica que permitiese integrar Google Earth

Engine o fuentes de imageriea equivalentes bajo condiciones compatibles con el análisis automático. Esto permitiría trabajar con imágenes más próximas a aquellas utilizadas durante el entrenamiento del modelo original y, por tanto, mejorar la coherencia entre el dominio de entrenamiento y el dominio real de explotación.

- En la gestión de modelos, resultaría interesante incorporar un sistema de comparación visual entre predicciones generadas por las distintas opciones cargadas. Además de mostrar ambas segmentaciones sobre una misma imagen, podría añadirse una máscara de diferencias que resaltase los píxeles que cambian de un modelo a otro, facilitando así una decisión más informada sobre qué modelo debe configurarse como activo.
- En el visor cartográfico, una mejora útil consistiría en implementar un buscador por zona o localización. De esta forma, el usuario podría introducir un municipio, una coordenada o una referencia geográfica y desplazarse automáticamente a esa región del mapa, agilizando la selección de áreas de interés y mejorando la usabilidad del flujo geoespacial.
- También se plantea como trabajo futuro la incorporación de un sistema de notificaciones por correo electrónico. Este mecanismo permitiría avisar al usuario cuando finalice el procesamiento de una colección, especialmente en casos donde la segmentación de todas las teselas requiera varios minutos. Con ello se mejoraría la retroalimentación del sistema y se evitaría que el usuario tuviera que permanecer consultando manualmente el estado del proceso.
- Otra posible ampliación sería enriquecer el sistema de descarga de resultados, permitiendo exportar no solo las imágenes y los ficheros JSON de trazas, sino también informes automáticos en PDF o CSV con estadísticas agregadas de la colección, como número de teselas procesadas, longitud aproximada de trazas, área segmentada, errores detectados y modelo utilizado.
- Desde el punto de vista algorítmico, sería conveniente incorporar un módulo de evaluación asistida que permitiese comparar las trazas generadas con anotaciones manuales de referencia. Esto permitiría calcular métricas como *IoU*, *Dice*, *accuracy*, *recall* o F_1 sobre imágenes concretas, facilitando la validación de nuevos modelos y el análisis cuantitativo de su rendimiento en escenarios reales.

- Finalmente, aunque el sistema ya soporta procesamiento concurrente controlado mediante varios *workers*, reclamación atómica de teselas y uso de SQLite como cola persistente, en escenarios de mayor carga podría evolucionarse hacia una cola de tareas especializada, como Redis o RabbitMQ. Esta migración permitiría incorporar mecanismos más avanzados de priorización, métricas de ejecución, paneles de monitorización, cancelaciones más finas y un escalado horizontal más robusto para contextos multiusuario o colecciones de gran tamaño.

Bibliografía

- [1] Agafonkin, Vladimir and contributors. Leaflet: An open-source javascript library for interactive maps. <https://leafletjs.com/>, 2026. Librería ligera para mapas interactivos en la web. Código abierto bajo licencia BSD-2. [Online; accedido 6-mayo-2026].
- [2] Kal Backman, Jared Wood, Maquel E. Brandimarti, Christine T. Beranek, and Alan Roff. Human inspired deep learning to locate and classify terrestrial and arboreal animals in thermal drone surveys. *Methods in Ecology and Evolution*, 16(5):1239–1254, 2025.
- [3] Vijay Badrinarayanan, Alex Kendall, and Roberto Cipolla. Segnet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2016. arXiv:1511.00561.
- [4] Saroj Bala, Kumud Arora, Jeevitha R, Rini Chowdhury, Prashant Kumar, and Shobana Nageswari C. A novel encoder decoder architecture with vision transformer for medical image segmentation. *Journal of Electronics, Electromedical Engineering, and Medical Informatics*, 7(1):176–186, 2025.
- [5] Zachary J Bortolot and Randolph H Wynne. Estimating forest biomass using small footprint lidar data: An individual tree-based approach that incorporates training data. *ISPRS Journal of Photogrammetry and Remote Sensing*, 59(6):342–360, 2005.
- [6] Han Cai, Junyan Li, Muyan Hu, Chuang Gan, and Song Han. Efficientvit: Lightweight multi-scale attention for high-resolution dense

- prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 17295–17305. IEEE/CVF, 2023.
- [7] Jeannine Cavender-Bares, Fabian D. Schneider, Maria João Santos, Amanda Armstrong, Ana Carnaval, George C. Hurtt, David Schimel, Philip A. Townsend, Susan L. Ustin, Adam M. Wilson, et al. Integrating remote sensing with ecology and evolution to advance biodiversity conservation. *Nature Ecology & Evolution*, 6:506–519, 2022.
 - [8] Centro Nacional de Información Geográfica (CNIG). Centro de descargas del cnig. <https://centrodedescargas.cnig.es/>, 2026. [Online; accessed 20-January-2026].
 - [9] Centro Nacional de Información Geográfica (CNIG). Ortofotos pnoa máxima actualidad. <https://centrodedescargas.cnig.es/CentroDescargas/ortofoto-pnoa-maxima-actualidad>, 2026. Mosai-cos de ortofotos más recientes disponibles del Plan Nacional de Orto-fotografía Aérea. Licencia compatible con CC-BY 4.0 y obligación de reconocimiento de origen y propiedad. [Online; accedido 6-mayo-2026].
 - [10] Zhaoxin Chang, Mengdi Xu, Yang Wei, Jianyuan Lian, Chunhui Zhang, and Cheng Li. UNeXt: An efficient network for the semantic segmen-tation of high-resolution remote sensing images. *Sensors*, 24(20):6655, 2024.
 - [11] François Chollet. Xception: Deep learning with depthwise separable convolutions, 2017.
 - [12] Copernicus Data Space Ecosystem. Copernicus browser. <https://browser.dataspace.copernicus.eu/>, 2026. [Online; accessed 19-January-2026].
 - [13] Ivica Dimitrovski, Vlatko Spasev, Suzana Loshkovska, and Ivan Kita-novski. U-net ensemble for enhanced semantic segmentation in remote sensing imagery. *Remote Sensing*, 16(12):2077, 2024.
 - [14] Vincent Dumoulin and Francesco Visin. A guide to convolution arith-metic for deep learning, 2018.
 - [15] José Francisco Díez-Pastor, Francisco Javier González-Moya, Pedro Latorre-Carmona, Francisco Javier Pérez-Barbería, Ludmila Kuncheva, Antonio Canepa-Oneto, Álgar Arnaiz-González, and César García-Osorio. Remote sensing colour image semantic segmentation of large herbivorous mammal trails. *International Journal of Remote Sensing*, 0(0):1–24, 2026.

- [16] Walid M. Elmessery, Dmitry V. Maklakov, Tarek M. El-Messery, Denis A. Baranenko, Javier Gutiérrez, Mohamed Y. Shams, Tarek Abd El-Hafeez, Shimaa Elsayed, Suliman K. Alhag, Farhat S. Moghanm, Mikhail A. Mulyukin, Yulia Y. Petrova, and Alaa E. Elwakeel. Semantic segmentation of microbial alterations based on SegFormer. *Frontiers in Plant Science*, 15:1352935, 2024.
- [17] Emergent Mind. Vision encoder: Architectures, training, efficiency, and applications. Technical report, 2025. Unpublished manuscript.
- [18] EmergentMind. Mobilenetv3: Efficient mobile CNN. Technical report, EmergentMind, 2024. Comprehensive review of MobileNetV3 architecture, NAS methodology, and hardware deployment.
- [19] European Space Agency (ESA). Snap (sentinel application platform) — download. <https://step.esa.int/main/download/snap-download/>, 2026. [Online; accessed 20-January-2026].
- [20] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 3 edition, 2008.
- [21] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [22] Google LLC. Google earth engine terms of service. <https://earthengine.google.com/terms/>, 2022. Términos de uso de Google Earth Engine. Última modificación: 29 de septiembre de 2022. [Online; accedido 6-mayo-2026].
- [23] Google LLC. Google earth engine. <https://earthengine.google.com/>, 2024. Plataforma web para análisis geoespacial a gran escala basada en computación en la nube. Acceso a catálogos de datos satelitales y herramientas de procesamiento.
- [24] Google LLC. Google earth pro (desktop) — versions and download. <https://www.google.com/earth/about/versions/>, 2026. [Online; accessed 20-January-2026].
- [25] Google LLC. Google maps platform terms of service. <https://cloud.google.com/maps-platform/terms>, 2026. Términos de servicio de Google Maps Platform, incluyendo restricciones sobre extracción, almacenamiento y creación de contenido derivado a partir de Google Maps Content. [Online; accedido 6-mayo-2026].

- [26] Noel Gorelick, Matt Hancer, Mike Dixon, Simon Ilyushchenko, David Thau, and Rebecca Moore. Google earth engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202:18–27, 2017.
- [27] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2024.
- [28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. IEEE, 2016.
- [29] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019. arXiv:1905.02244.
- [30] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4700–4708, 2017.
- [31] Andreea Iana, Goran Glavaš, and Heiko Paulheim. Peeling back the layers: An in-depth evaluation of encoder architectures in neural news recommenders. In *Proceedings of INRA 2024: 12th International Workshop on News Recommendation*, pages 1–10. ACM, 2024. arXiv:2410.01470.
- [32] Instituto Geográfico Nacional (IGN) de España. Plan nacional de ortofotografía aérea (pnoa) - ortofotografías rgb de españa. <https://pnoa.ign.es/>. Ortofotografías aéreas de alta resolución disponibles para análisis geoespacial; datos accesibles también a través del Earth Engine Data Catalog.
- [33] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1125–1134, 2017.
- [34] Ankang Ji, Alvin Wei Ze Chew, Xiaolong Xue, and Limao Zhang. An encoder-decoder deep learning method for multi-class object segmentation from 3d tunnel point clouds. *Automation in Construction*, 137:104187, 2022.

- [35] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. Transformers in vision: A survey. *ACM Computing Surveys*, 54(10s):1–41, 2022.
- [36] Miao Liao, Hongliang Tang, Xiong Li, P. Vijayakumar, Varsha Arya, and Brij B. Gupta. A lightweight network for abdominal multi-organ segmentation based on multi-scale context fusion and dual self-attention. *Information Fusion*, 108:102401, 2024.
- [37] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 936–944. IEEE, 2017.
- [38] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11976–11988. IEEE/CVF, 2022.
- [39] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3431–3440, 2015.
- [40] Gregory J. McDermid, Irina Terenteva, and Xue Yan Chan. Mapping trails and tracks in the boreal forest using lidar and convolutional neural networks. *Remote Sensing*, 17(9):1539, 2025.
- [41] Shervin Minaee, Yuri Boykov, Fatih Porikli, Antonio Plaza, Nasser Kehtarnavaz, and Demetri Terzopoulos. Image segmentation using deep learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3523–3542, 2022.
- [42] Kathrin¹ Müller, Leonie¹ Sonntag, Marin Bevanda, Alejandro J. Castañeda-Gómez, Martin Wegmann, Benjamin Wigley, Corli Coetsee, David Klein, Stefan Dech, and Jonas Schwalb-Willmann. Automated segmentation of animal paths in uas imagery using deep learning — a case study from the kruger national park. *Ecological Informatics*, page 103570, 2025. ¹Autores contribuyentes iguales.
- [43] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010.

- [44] NASA Earthdata. Nasa worldview. <https://worldview.earthdata.nasa.gov/>, 2026. [Online; accessed 20-January-2026].
- [45] Asif Ahmed Nelay and Maxime Turgeon. A comprehensive study of auto-encoders for anomaly detection: Efficiency and trade-offs. *Machine Learning with Applications*, 17:100572, 2024.
- [46] NextGIS. Quickmapservices (qgis plugin). https://plugins.qgis.org/plugins/quick_map_services/, 2026. [Online; accessed 19-January-2026].
- [47] OpenStreetMap contributors. Openstreetmap. <https://www.openstreetmap.org/>, 2026. Mapa mundial editable colaborativamente por la comunidad. Datos geoespaciales accesibles bajo licencia ODbL. [Online; accedido 6-mayo-2026].
- [48] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [49] Nathalie Pettorelli, William F. Laurance, Timothy G. O'Brien, Martin Wegmann, Harini Nagendra, and Woody Turner. Satellite remote sensing for applied ecologists: opportunities and challenges. *Journal of Applied Ecology*, 51(4):839–848, 2014.
- [50] David M. W. Powers. Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation, 2020.
- [51] QGIS Association. Qgis: A free and open source geographic information system. <https://qgis.org/>, 2026. [Online; accessed 19-January-2026].
- [52] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos, 2024.
- [53] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015.

- [54] Mohammed Ali Saleh, Mohamed Abdouh, and Mohamed K. Ramadan. A novel method for improving baggage classification using a hyper model of fusion of densenet-161 and efficientnet-b5. *Big Data and Cognitive Computing*, 8(10), 2024.
- [55] Sentinel Hub (Sinergise). Sentinel hub eo browser. <https://apps.sentinel-hub.com/eo-browser/>, 2026. [Online; accessed 19-January-2026].
- [56] Sentinel Hub (Sinergise). Sentinel hub qgis plugin. <https://www.sentinel-hub.com/develop/integrate/desktopqgis/qgis-plugin/>, 2026. [Online; accessed 19-January-2026].
- [57] ShadeCoder. Resnet-34: A comprehensive guide for 2025, 2026. [Online; accessed 02-February-2026].
- [58] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009.
- [59] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.
- [60] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. In *Proceedings of the Deep Learning Workshop at the 32nd International Conference on Machine Learning (ICML)*. arXiv preprint arXiv:1505.00387, 2015.
- [61] Keenan Stears, Melissa H. Schmitt, Mike J. Peel, Tsumbedzo Ramalevha, Douglas J. McCauley, Dave I. Thompson, and Deron E. Burkepile. Remotely-sensed game trails are a behavioral footprint that explains patterns of herbivore habitat use. *Ecology and Evolution*, 15(1):e70792, 2025.
- [62] Yibo Sun, Zhe Sun, and Weitong Chen. The evolution of object detection methods. *Engineering Applications of Artificial Intelligence*, 133:108458, 2024.
- [63] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alex Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning, 2016.

- [64] Abdel Aziz Taha and Allan Hanbury. Metrics for evaluating 3d medical image segmentation: analysis, selection, and tool. *BMC Medical Imaging*, 15(1):29, 2015.
- [65] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 6105–6114. PMLR, 2019.
- [66] Woody Turner, Stephen Spector, Nate Gardiner, Matt Fladeland, Emily Sterling, and Marc Steininger. Remote sensing for biodiversity science and conservation. *Trends in Ecology & Evolution*, 18(6):306–314, 2003.
- [67] U.S. Geological Survey (USGS). Usgs earthexplorer. <https://earthexplorer.usgs.gov/>, 2026. [Online; accessed 19-January-2026].
- [68] Pavan Kumar Anasosalu Vasu, James Gabriel, Jeff Zhu, Oncel Tuzel, and Anurag Ranjan. Mobileone: An improved one millisecond mobile backbone, 2023.
- [69] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- [70] Zihui Wang, Yuan Jiang, Abdoulaye Baniré Diallo, and Steven W. Kembel. Deep learning- and image processing-based methods for automatic estimation of leaf herbivore damage. *Methods in Ecology and Evolution*, 15(4):732–743, 2024.
- [71] Tete Xiao, Yingcheng Liu, Bolei Zhou, Yuning Jiang, and Jian Sun. Unified perceptual parsing for scene understanding. In *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 2018.
- [72] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [73] Weihao Yu and Xinchao Wang. Mambaout: Do we really need mamba for vision?, 2024.
- [74] et al. Zhang. Paper published in automation in construction. *Automation in Construction*, 2022. Elsevier article extracted from PDF 1-s2.0-S0926580522000607.

- [75] T. Y. Zhang and C. Y. Suen. A fast parallel algorithm for thinning digital patterns. *Communications of the ACM*, 27(3):236–239, 1984.
- [76] Hengshuang Zhao, Jianping Shi, Xiaojuan Qi, Xiaogang Wang, and Jiaya Jia. Pyramid scene parsing network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2881–2890. IEEE, 2017.